



# Tutorial

## Modulação QPSK no Simulink e Geração de VHDL para FPGA

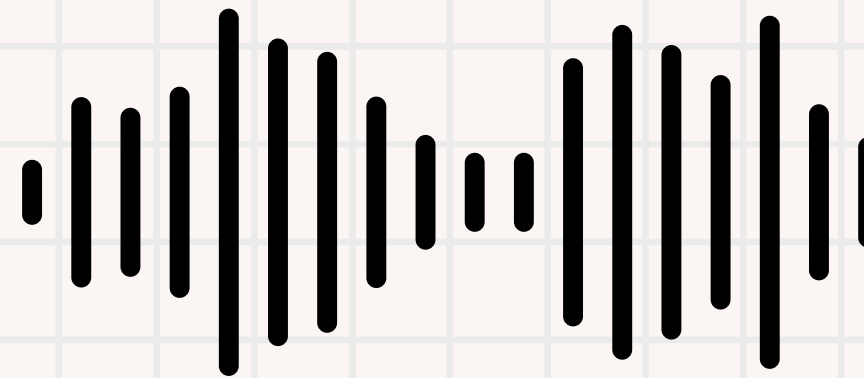
Laboratórios Didáticos para Ensino de Sistemas de  
Comunicação em FPGA

**Materiais**

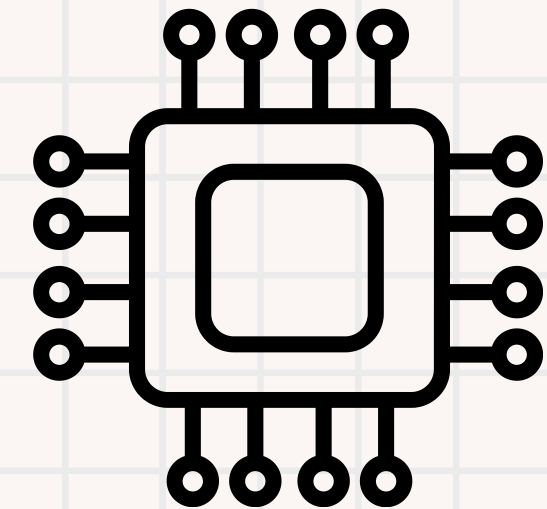
# Pré-requisitos



**MATLAB + Simulink  
instalados**



**Conceito rápido da  
QPSK**



**Quartus Prime  
(Intel/Altera FPGA) +  
Modelsim**

# Simulink + VHDL

A modulação QPSK (Quadrature Phase Shifting Keying) transmite 2 bits por símbolo, combinando sinais seno e cosseno. No Simulink, conseguimos modelar facilmente o modulador, validar por simulação e depois gerar o código VHDL para implementação em FPGA.



## Introdução

# Fluxo de trabalho com Quartus + ModelSim



Modelagem no Simulink (modulador QPSK)



HDL Coder gera o VHDL



ModelSim simula o VHDL



Desenvolvimento

# Passo a Passo





Desenvolvimento

# Simulink

## Arquivo do projeto

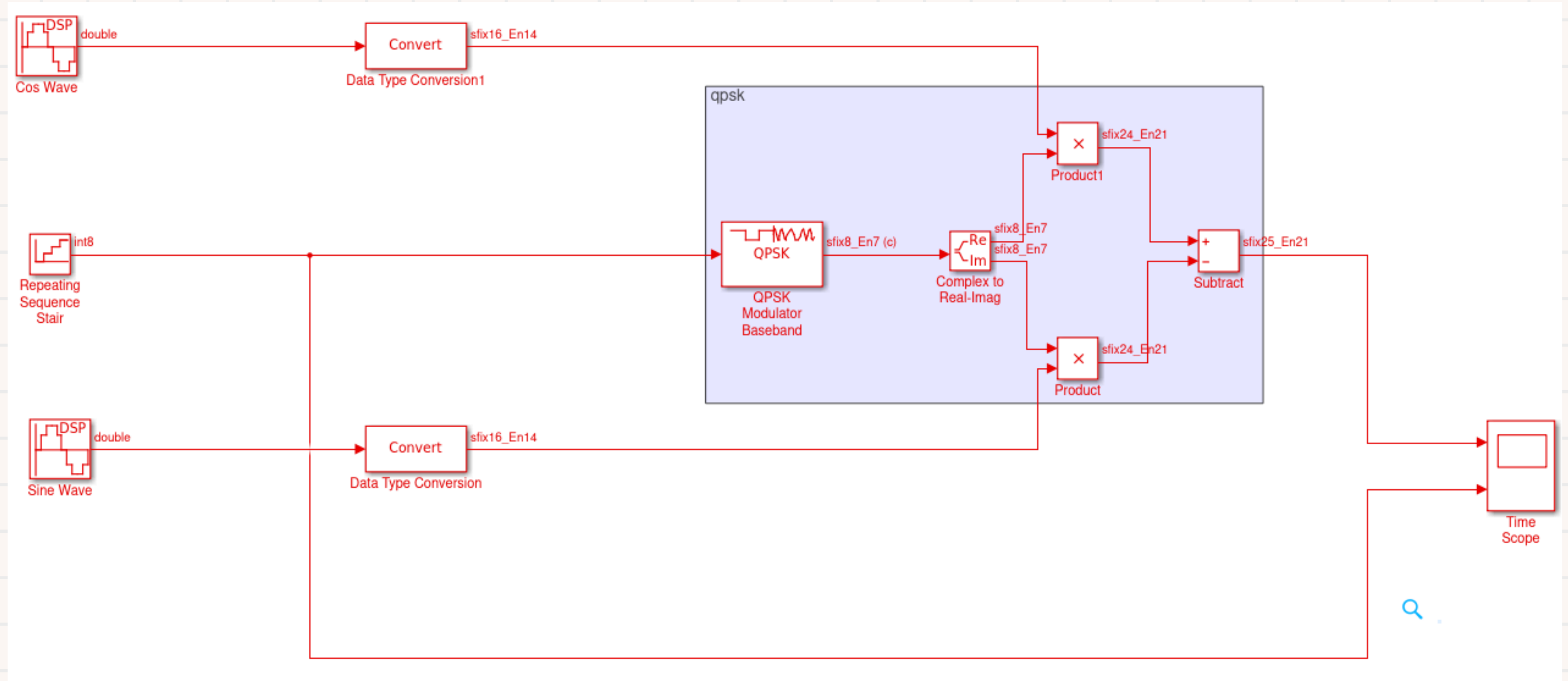
Para facilitar, é disponibilizado arquivo do modelo em Simulink no Github:



[Link Download](#)

## Desenvolvimento

# Modulação QPSK - Sistema





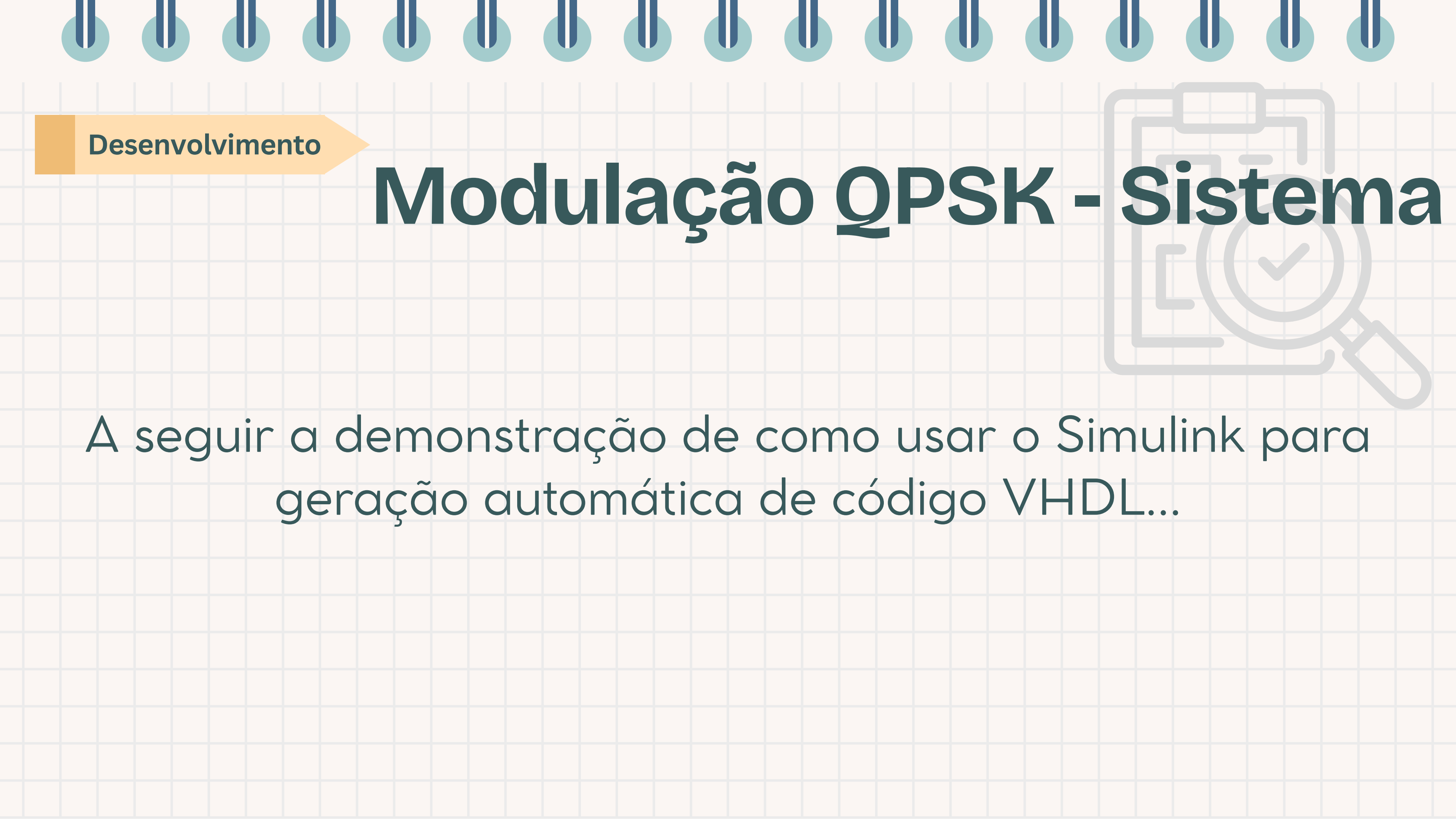
Desenvolvimento

# Modulação QPSK - Sistema



## No projeto você já encontra:

- Fonte de bits (Random Integer Generator) → Gerando números de 0 à 3
- Mapeamento QPSK (QAM Modulator Baseband com  $M=4$ )
- Separação I/Q (Complex to Real-Imag)
- Sinais( para gerar seno/cosseno) → Como portadora
- Multiplicação e subtração → sinal QPSK modulado
- Blocos de conversão para Fixed-Point (compatível com HDL Coder)
- Time Scope para visualização



Desenvolvimento

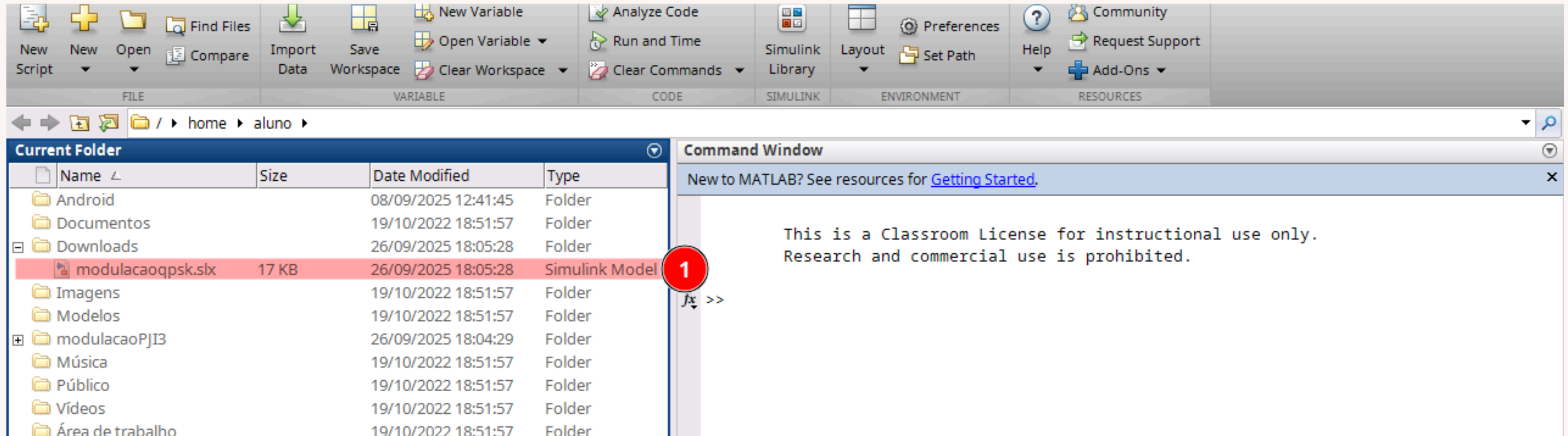
# Modulação QPSK - Sistema



A seguir a demonstração de como usar o Simulink para geração automática de código VHDL...



## Desenvolvimento



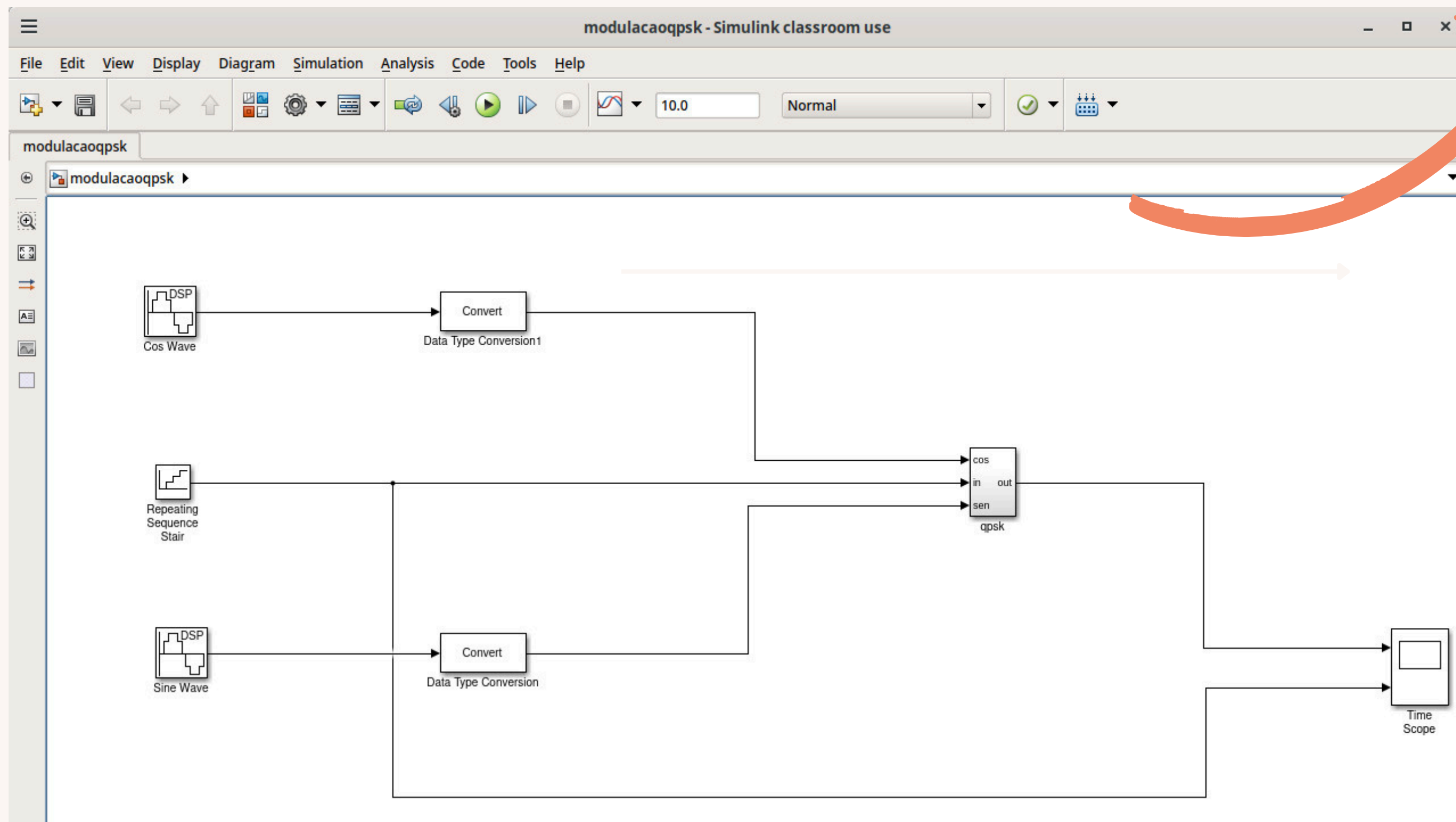
Abra o Matlab

Vá até ao diretório onde foi salvado o arquivo **modulacaoqpsk.slx**

1 - Dê um duplo click

## Desenvolvimento

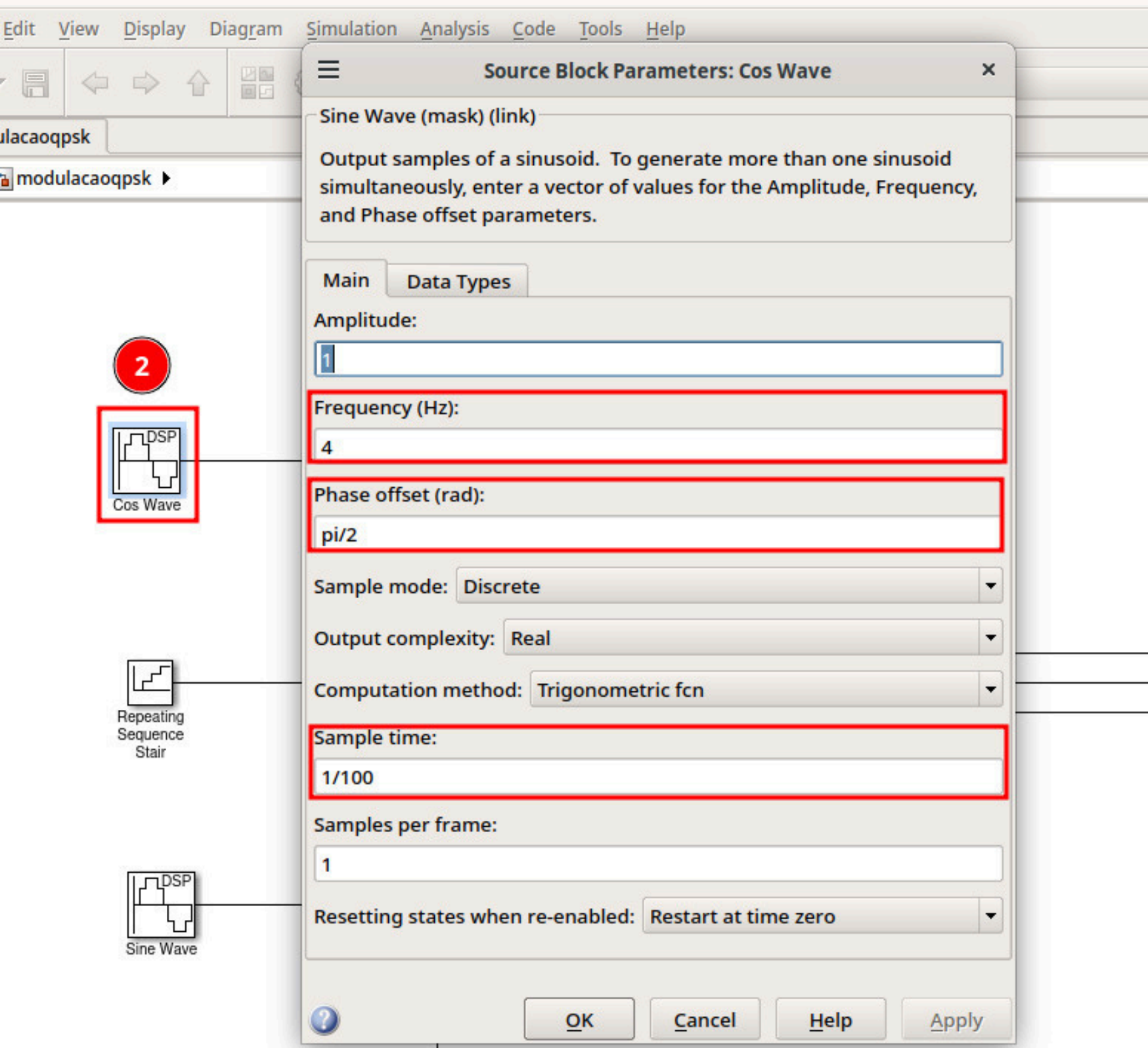
## modulacaoqpsk - Simulink classroom use



Abrirá essa aba  
do simulink



## Desenvolvimento



- Note que existe 3 entradas

Cosseno

Seno

Entrada aleatória de 0 a 3

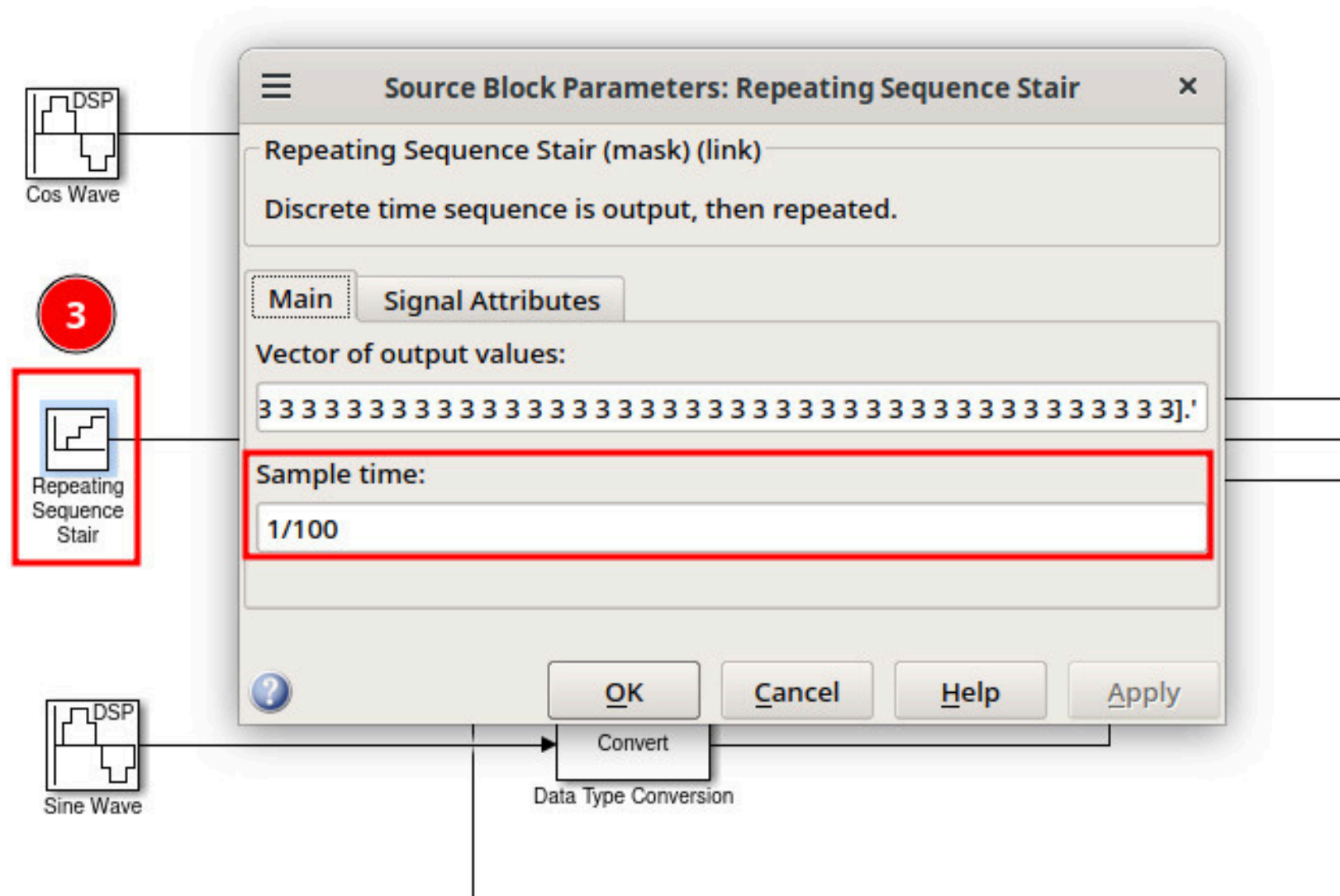
2 - Como não existe o bloco de cossenoide, o bloco do seno foi alterado a “Phase offset (rad)” para  $\pi/2$

- Frequência alterada para 4 hz

- Discretizado para amostras de  $1/100$



## Desenvolvimento

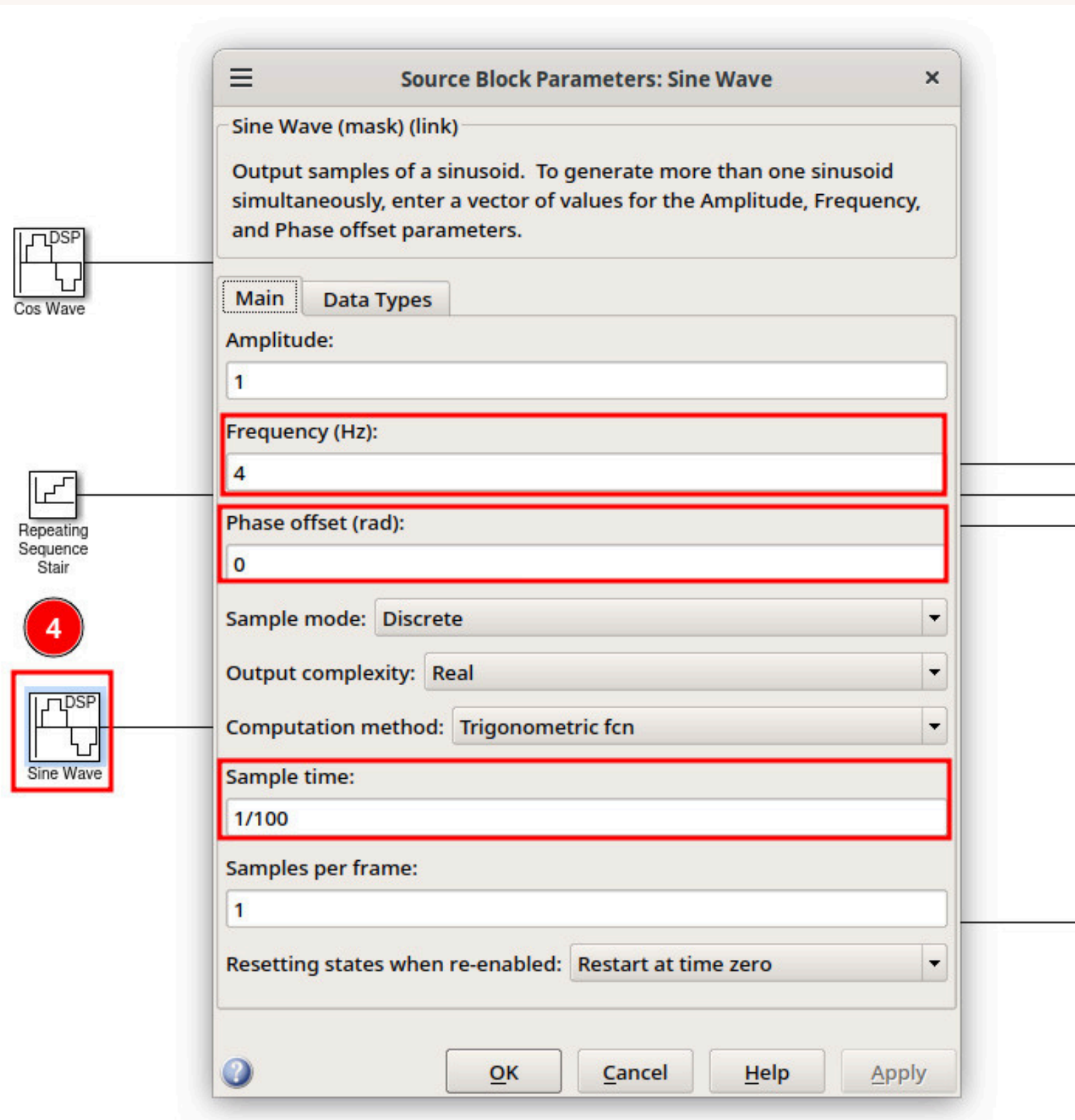


Todos os sinais devem ter o mesmo valor de amostragem: “sample time”!



3 - Discretizado para amostras de 1/100

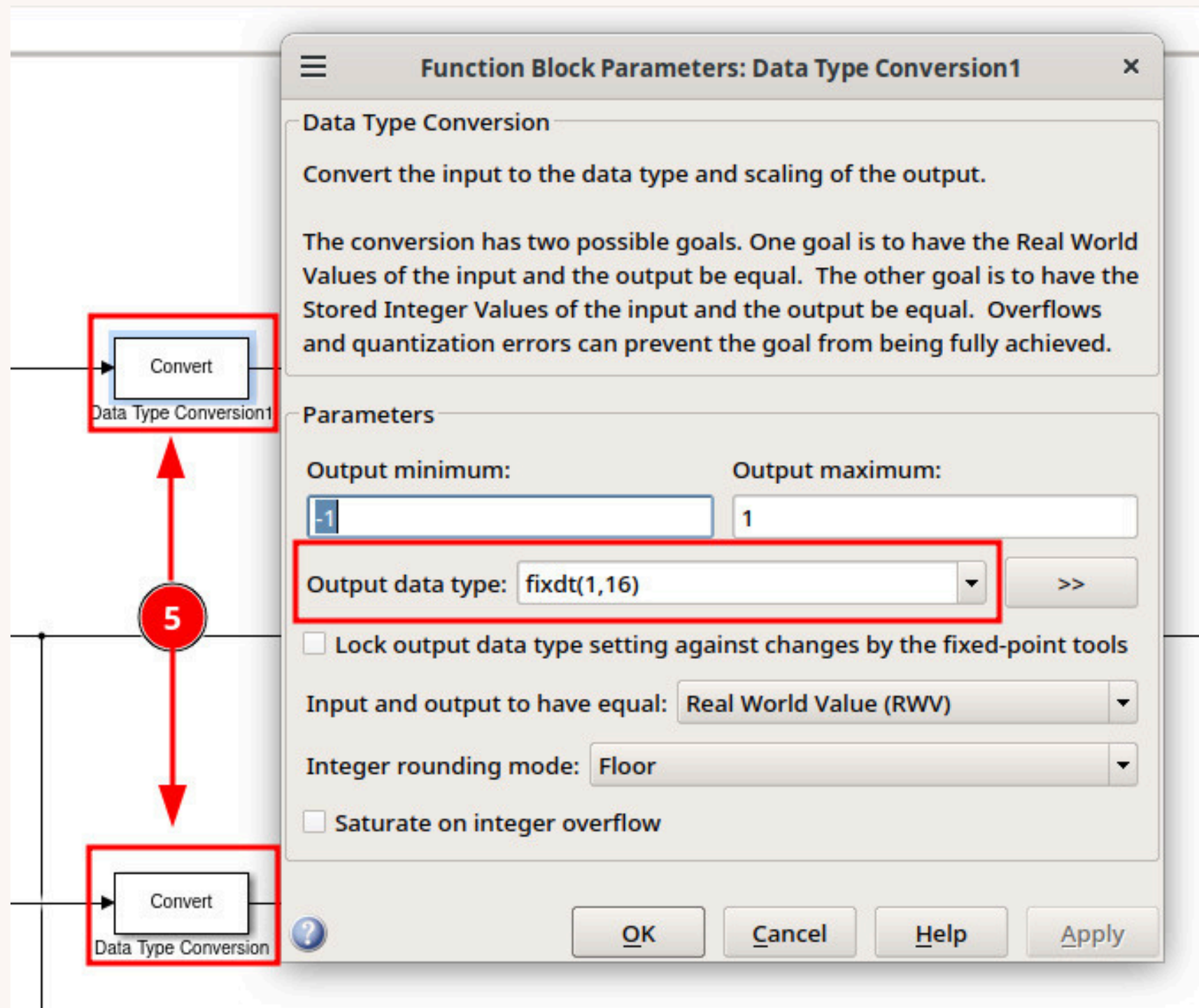
## Desenvolvimento



4 - Note, que o bloco seno, o “Phase offset (rad)” não foi alterado, permanecendo em **Zero**

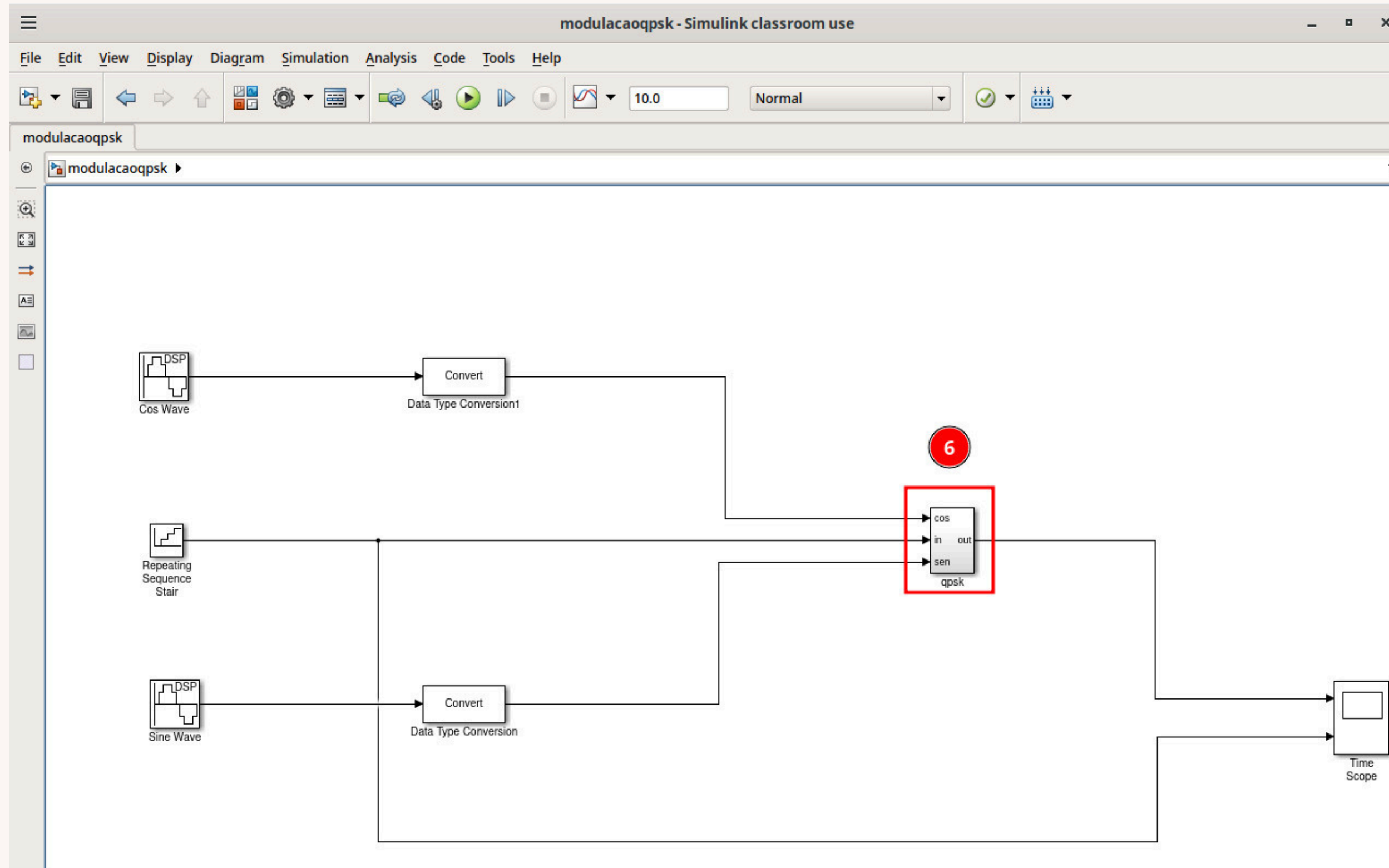
- Frequência alterada para **4 hz**
- Discretizado para amostras de **1/100**

## Desenvolvimento



5 - Como a saída dos sinais seno/cosseno são do tipo “double”, é necessário um conversor para `fixdt(1,16)`

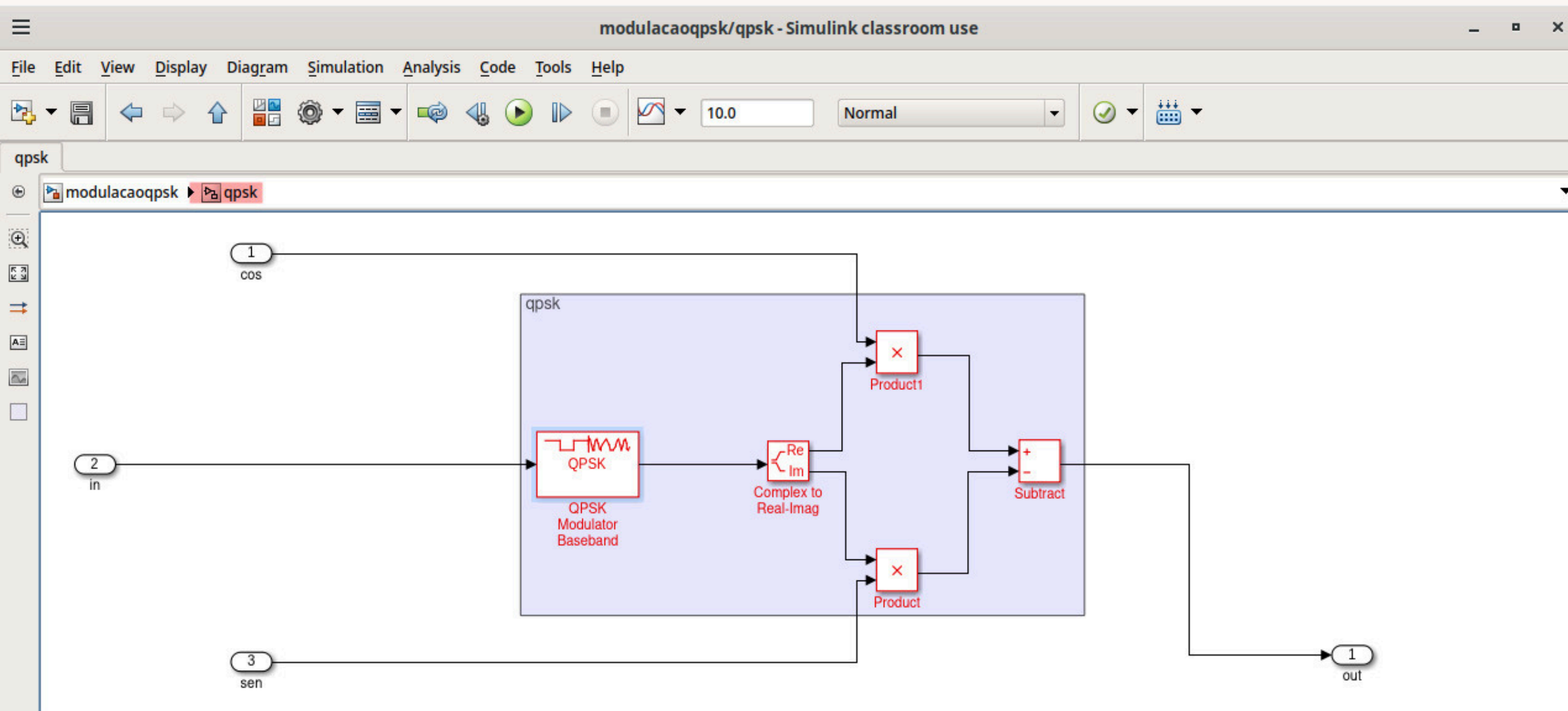
## Desenvolvimento



6 - Dando um duplo click...

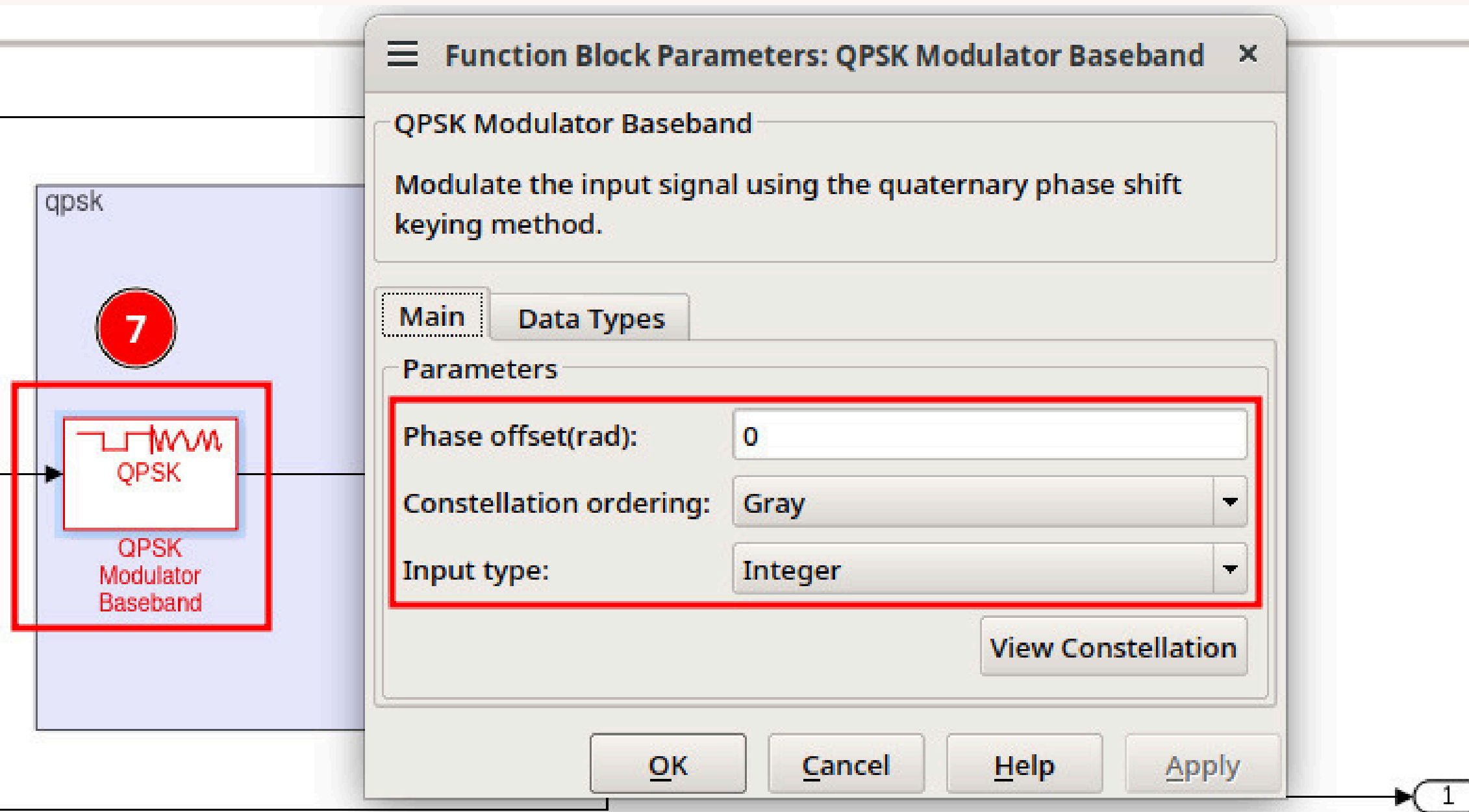


## Desenvolvimento



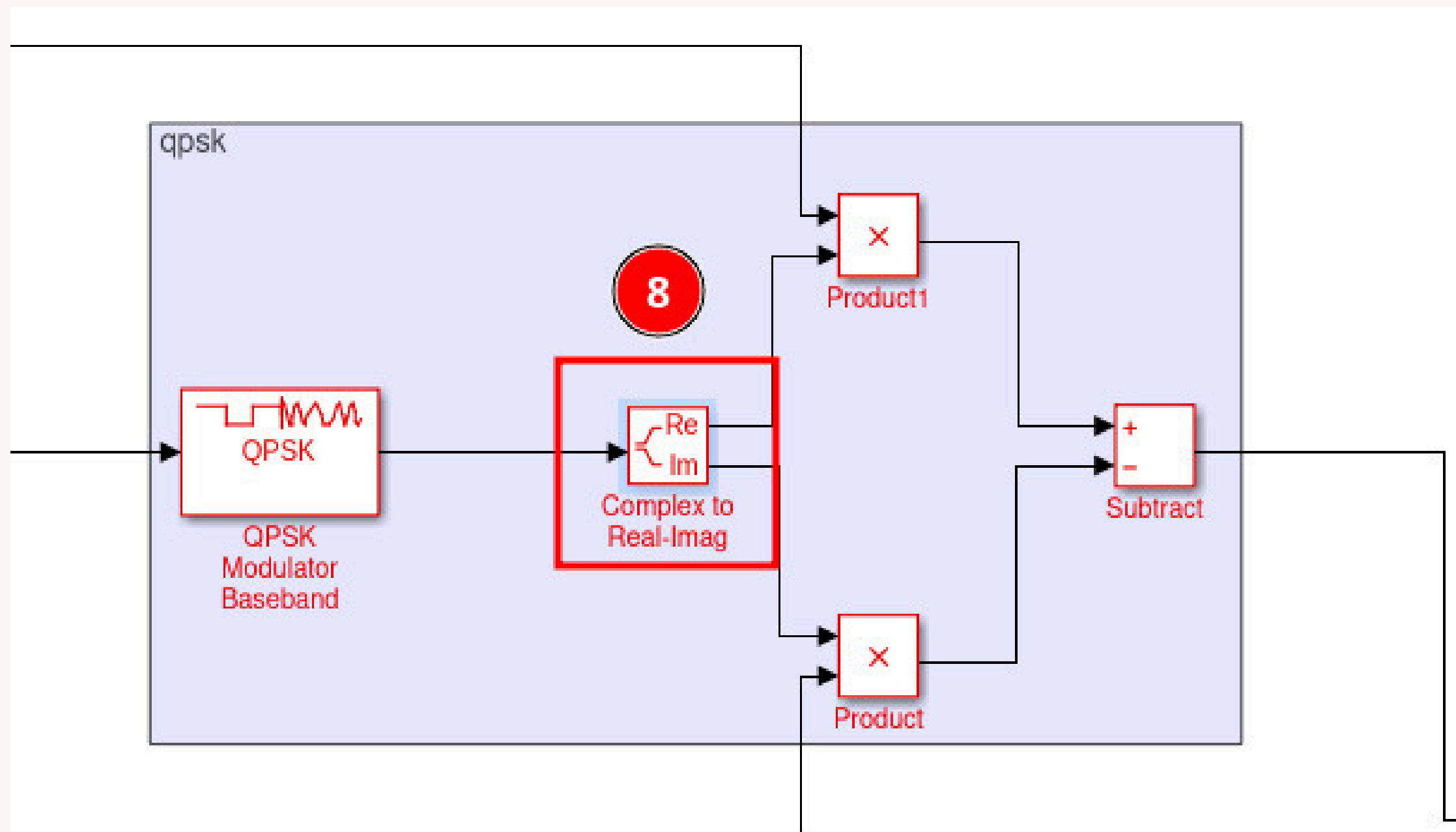
Abrirá o  
subsistema **qpsk**  
(note que há  
uma **hierarquia**)

## Desenvolvimento



7 - “Phase offset (rad)” foi alterado para **zero**, de modo que os ângulos no QPSK sejam  $0^\circ$ ,  $90^\circ$ ,  $180^\circ$  e  $270^\circ$

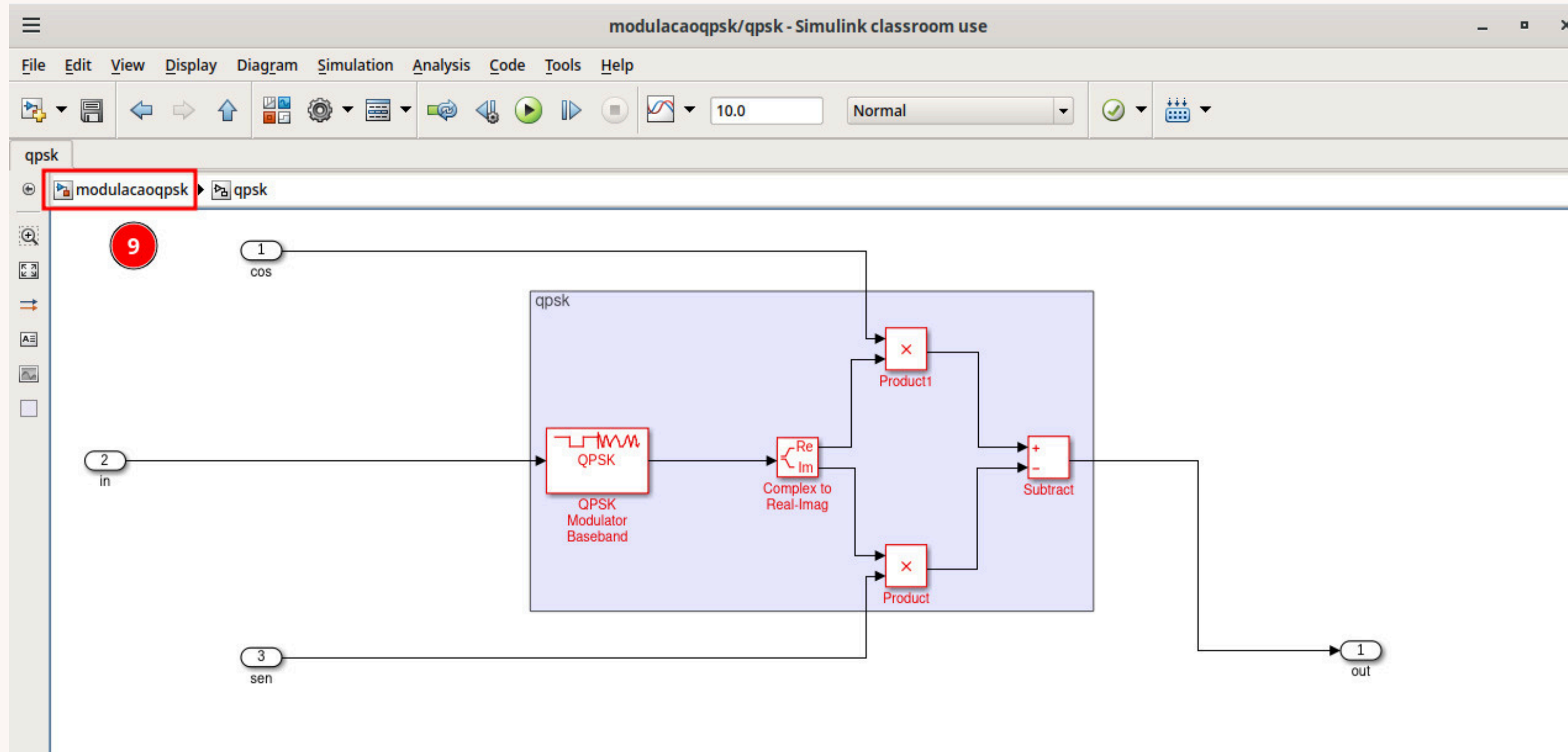
## Desenvolvimento



8 - O bloco destacado separa o sinal em componentes de **fase** e **quadratura**, que são então multiplicadas por seno e cosseno antes de serem processadas pelo subtrator. Seguindo a fórmula:

$$s(t) = I(t) \cdot \cos(2\pi f_c t) - Q(t) \cdot \sin(2\pi f_c t)$$

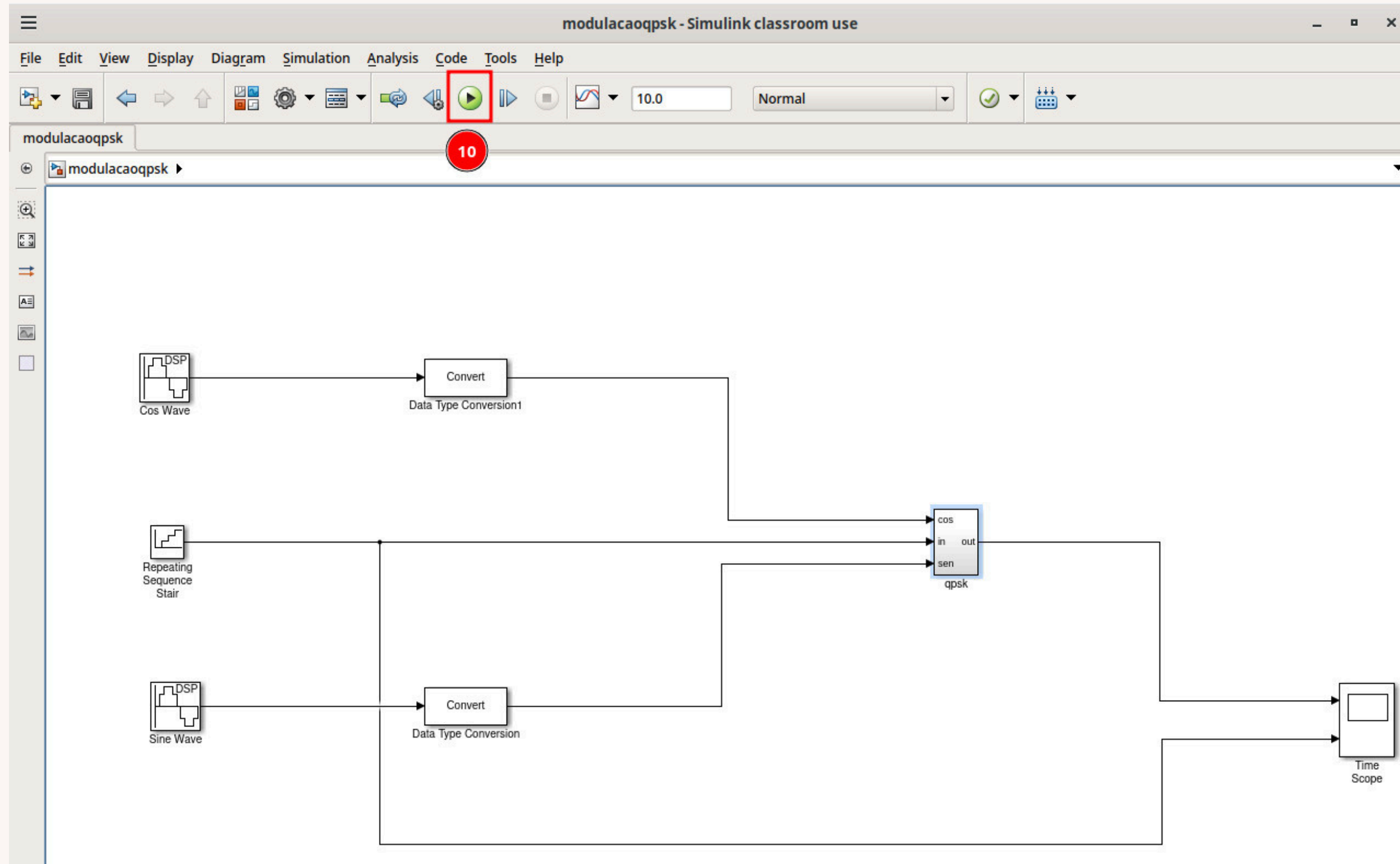
## Desenvolvimento



9 - Volte para o topo...

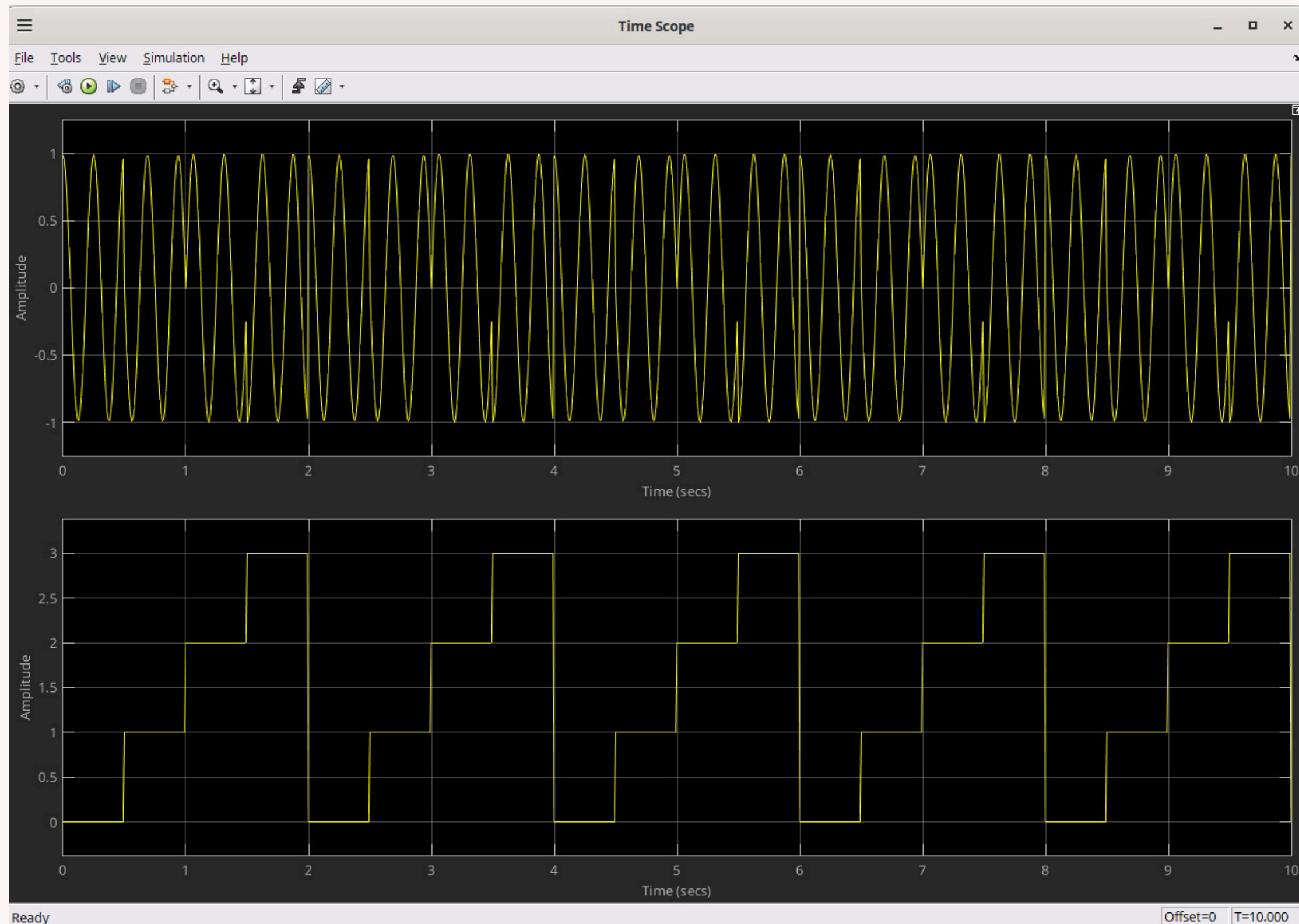


## Desenvolvimento



10 - Clique em **run**

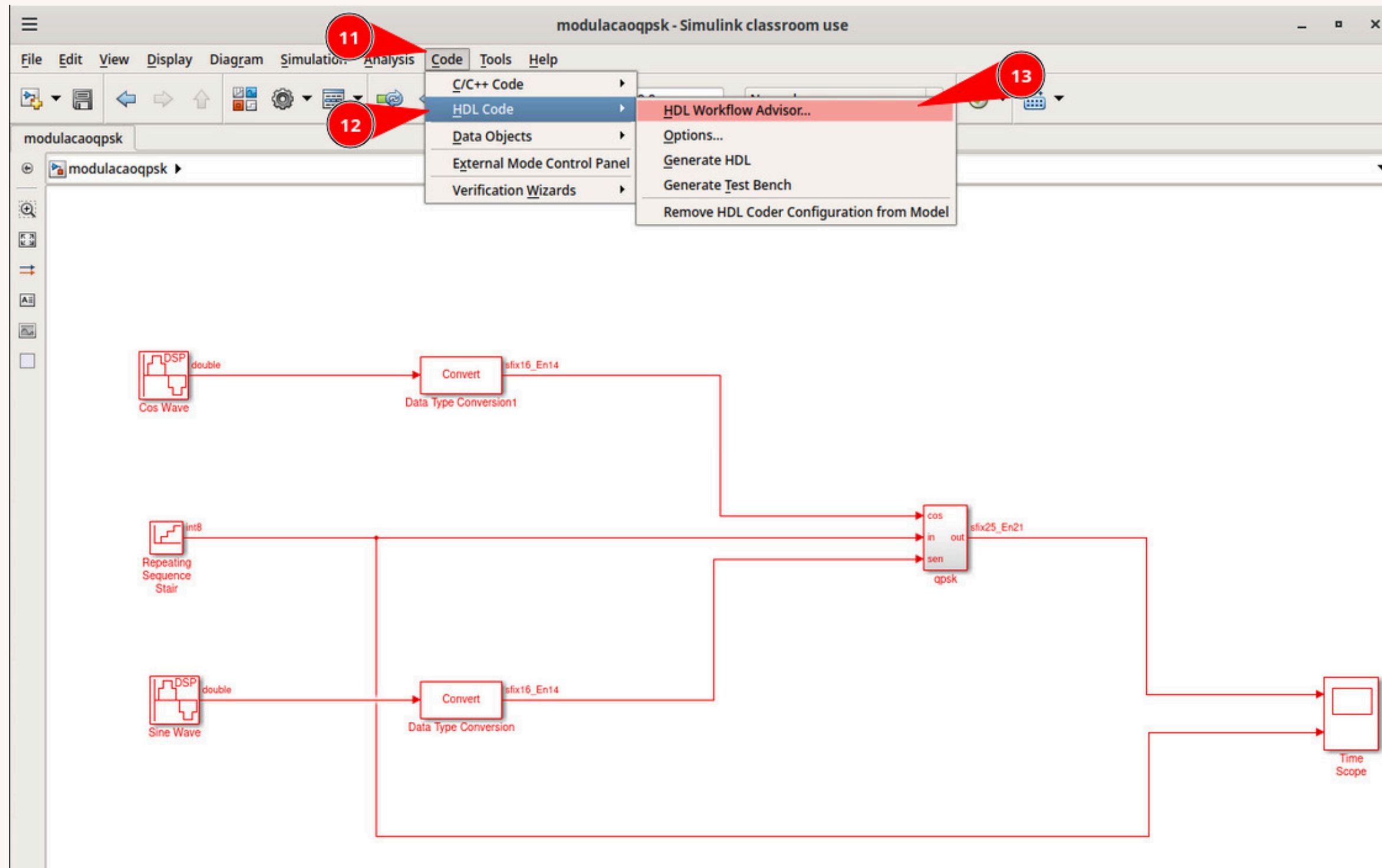
## Desenvolvimento



Abrirá o Time Scope...

Note as mudanças de fase a cada mudança de nível (em cada 0,5 seg.)

## Desenvolvimento



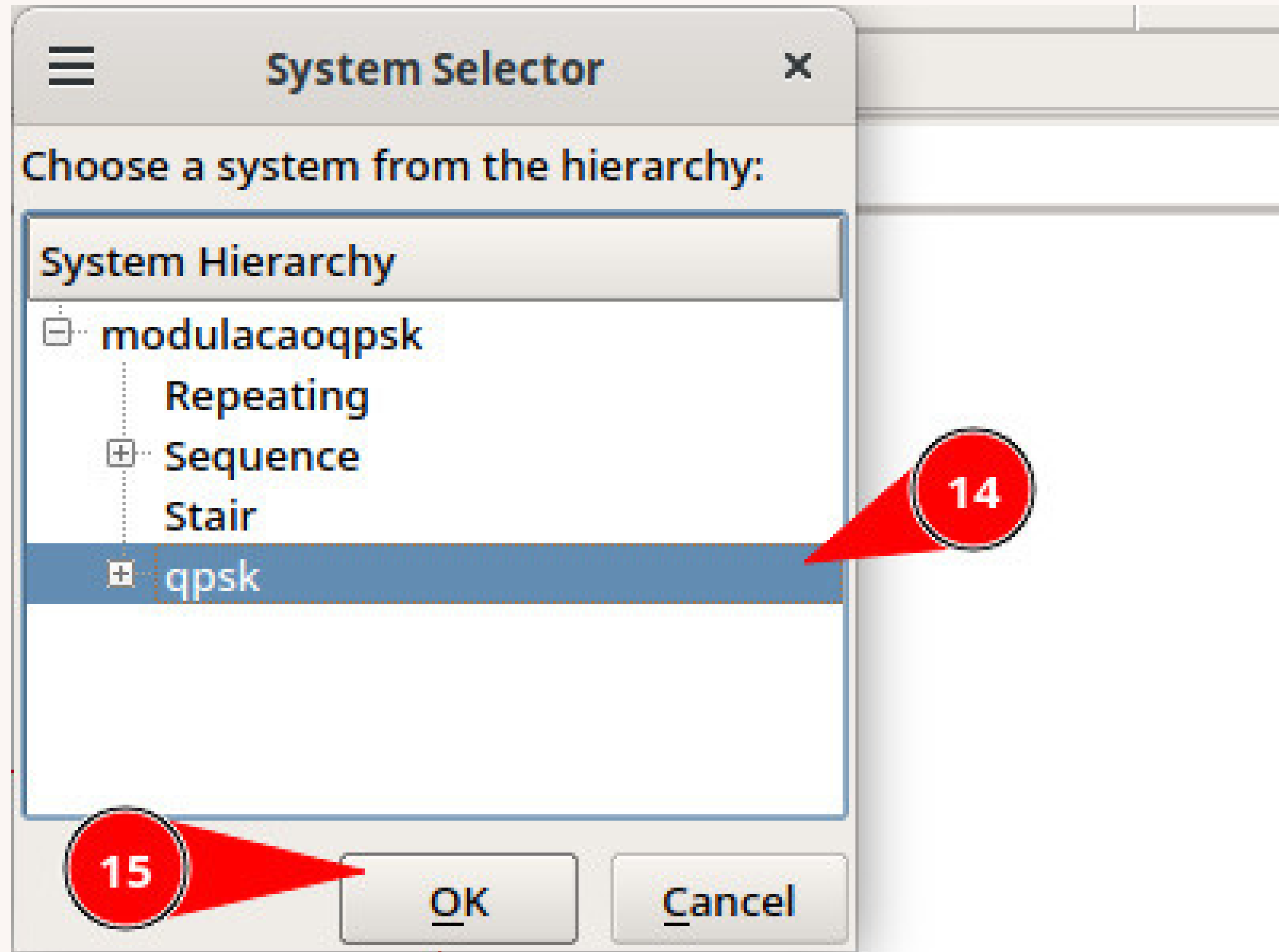
Feche o Time Scope.

11- Clique em **Code**

12 - Clique em **HDL Code**

13 - Clique em **HDL Workflow Advisor...**

## Desenvolvimento



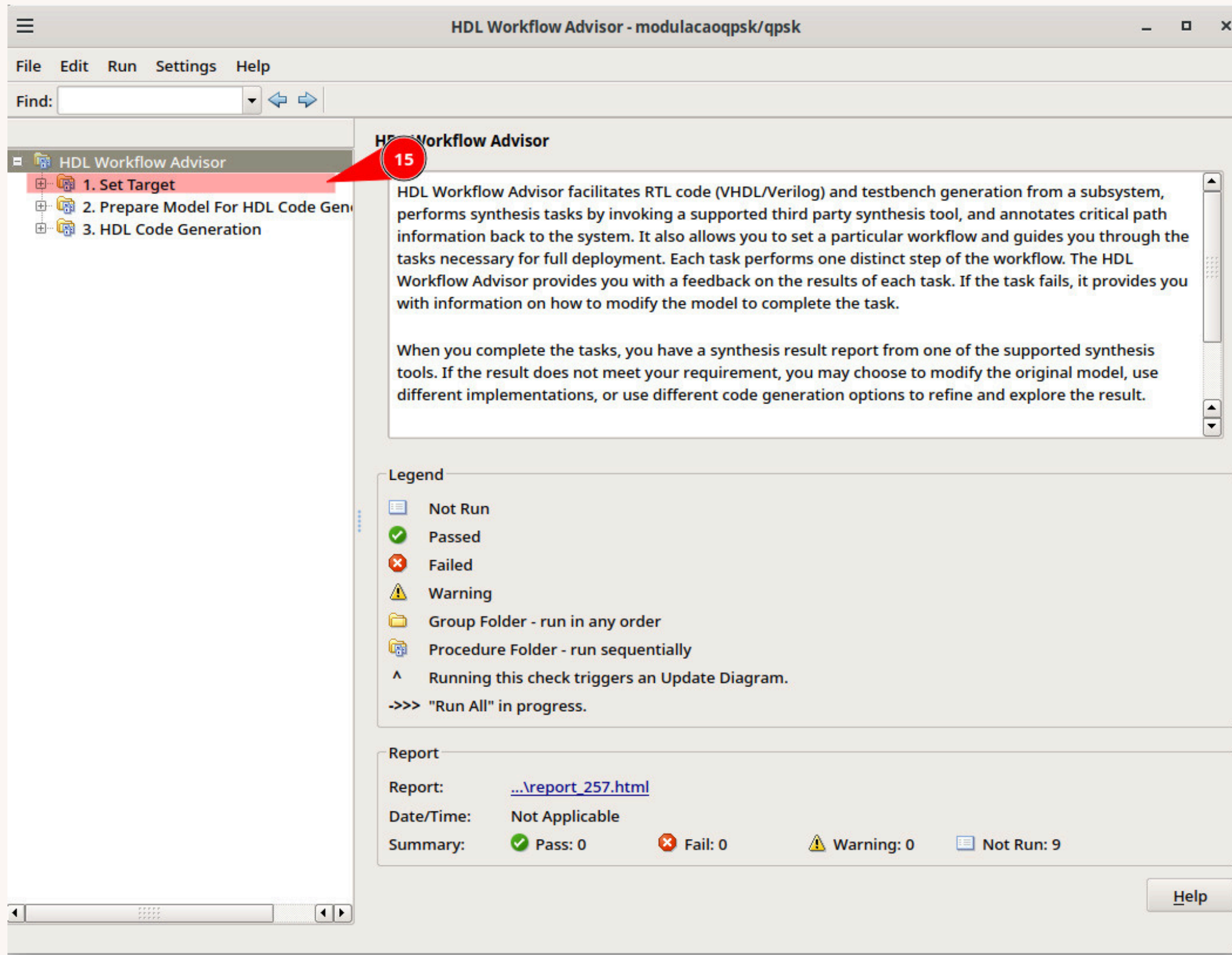
Abrirá outra aba...

14 - Selecione apenas o **qpsk**

15 - Clique em **Ok**



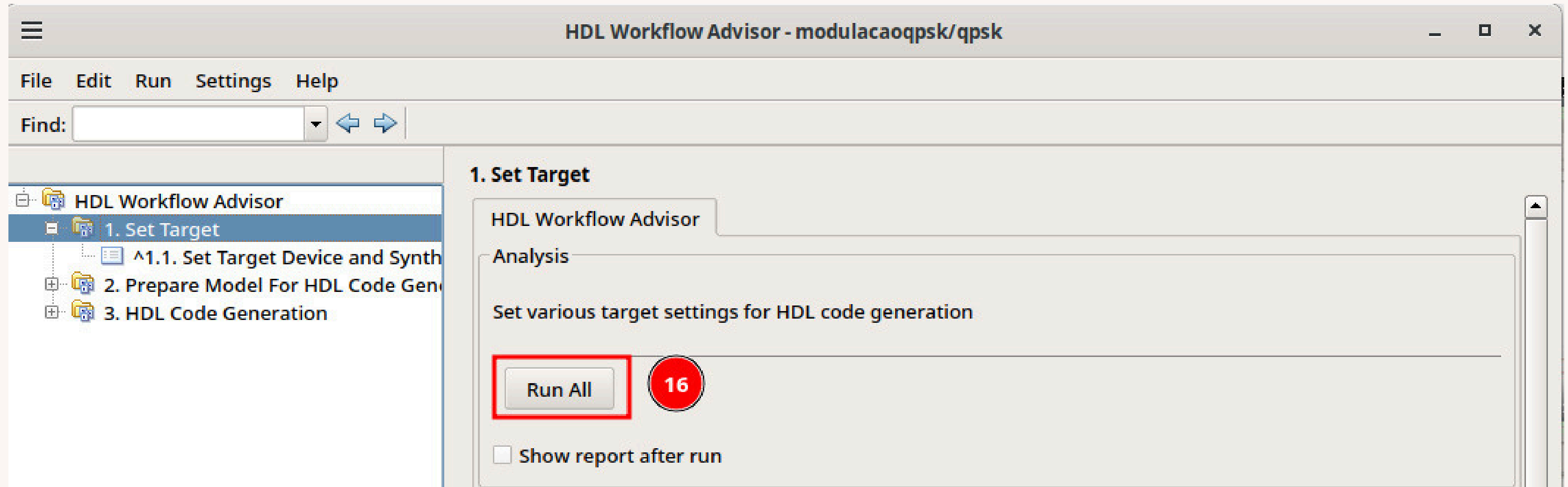
## Desenvolvimento



Abrirá outra aba...

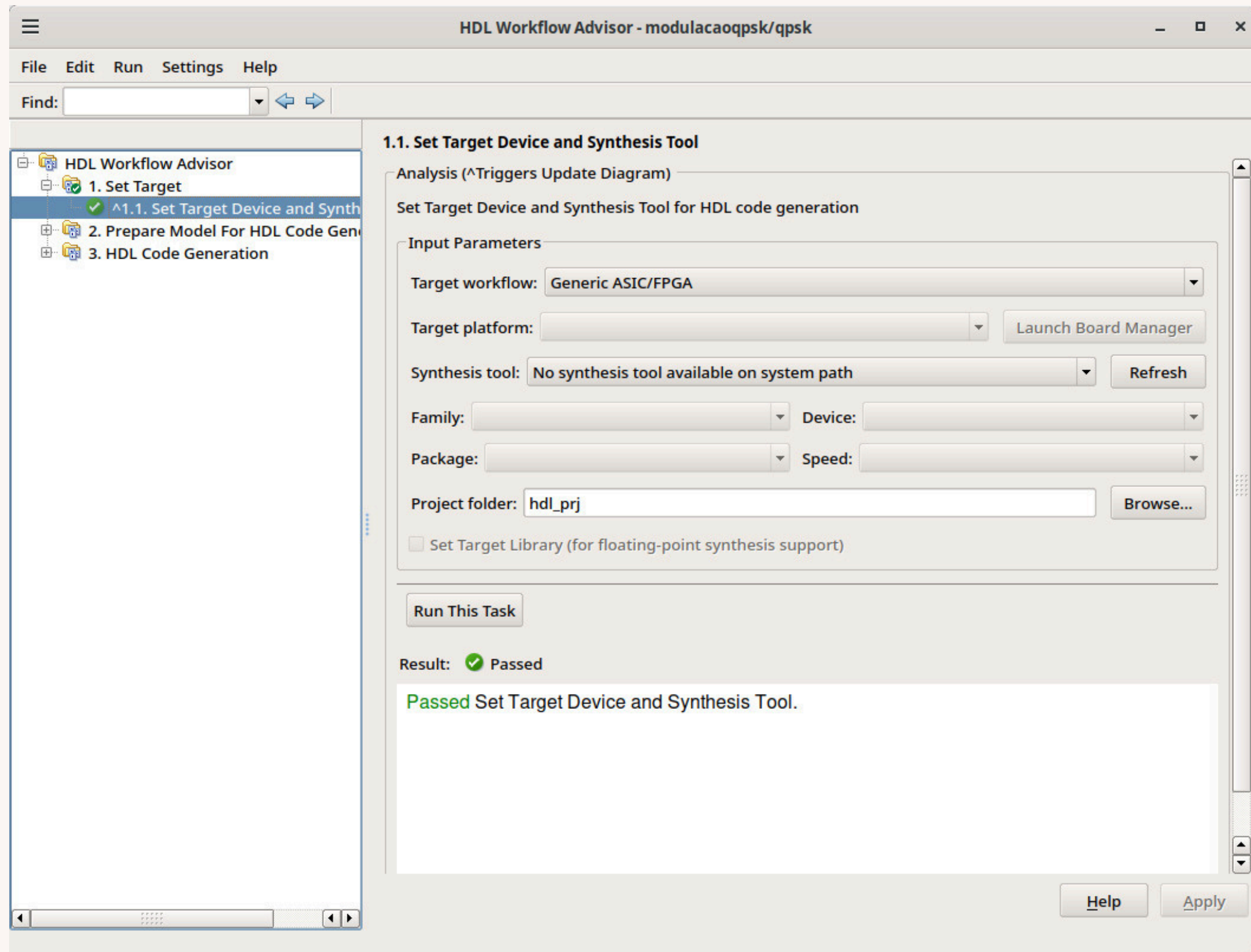
Clique em **Set Target**

## Desenvolvimento



16 - Clique em **Run All**

## Desenvolvimento



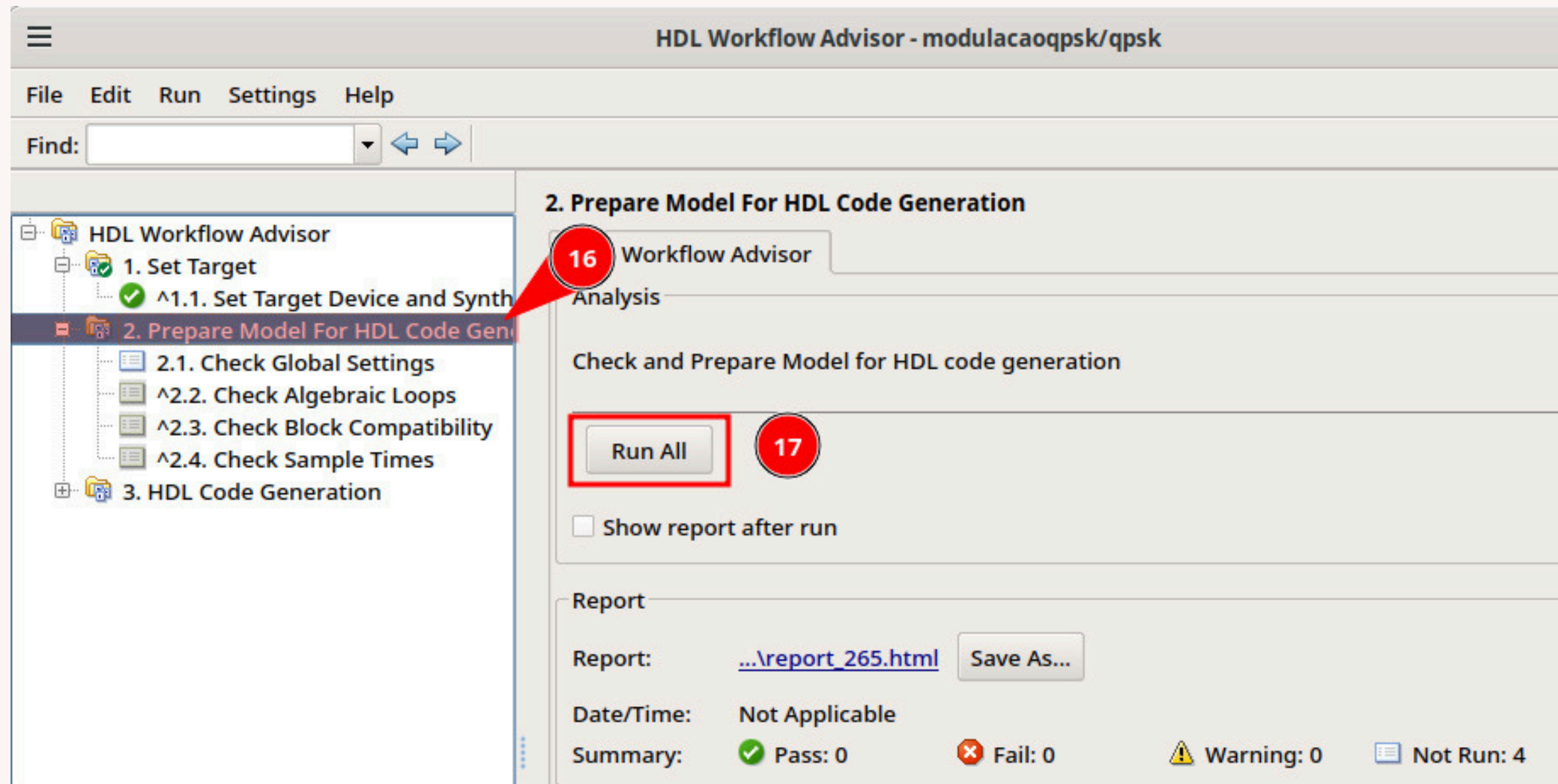
Ficará uma indicação de  
“passed”...

**NOTE QUE:** os arquivos  
gerados ficarão em **hdl\_prj**





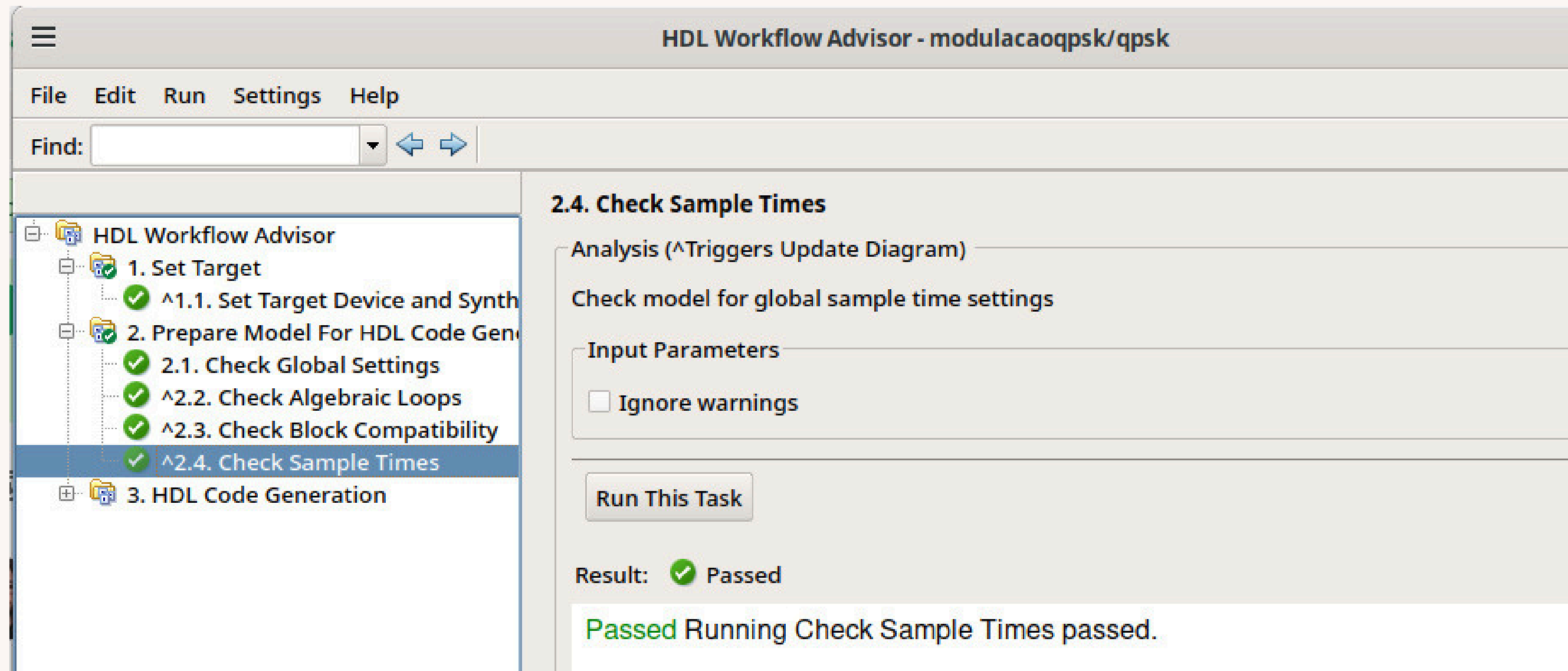
## Desenvolvimento



Clique no segundo passo

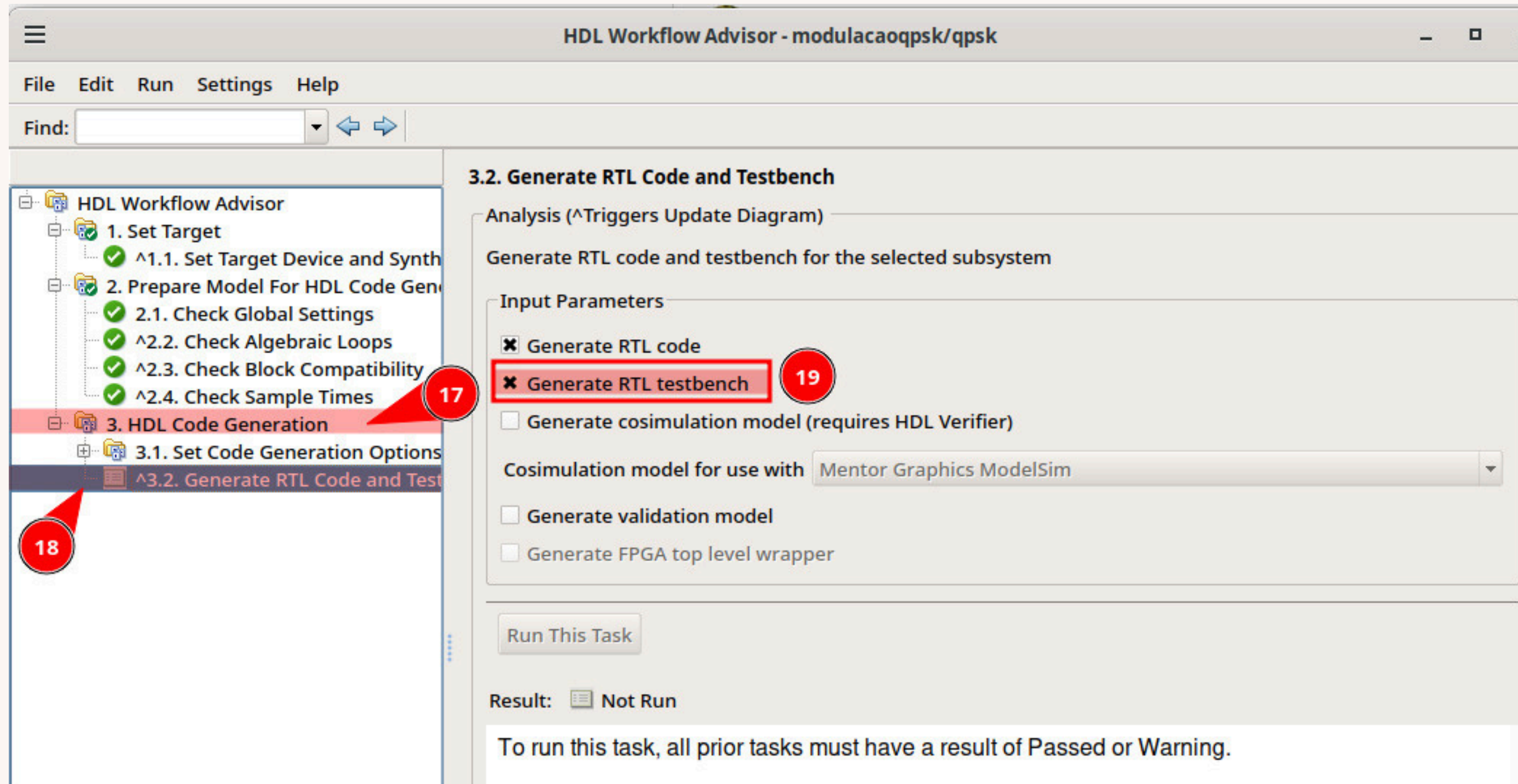
17 - Clique em **Run All**

## Desenvolvimento



Ficará uma  
indicação de  
“Passed”...

## Desenvolvimento

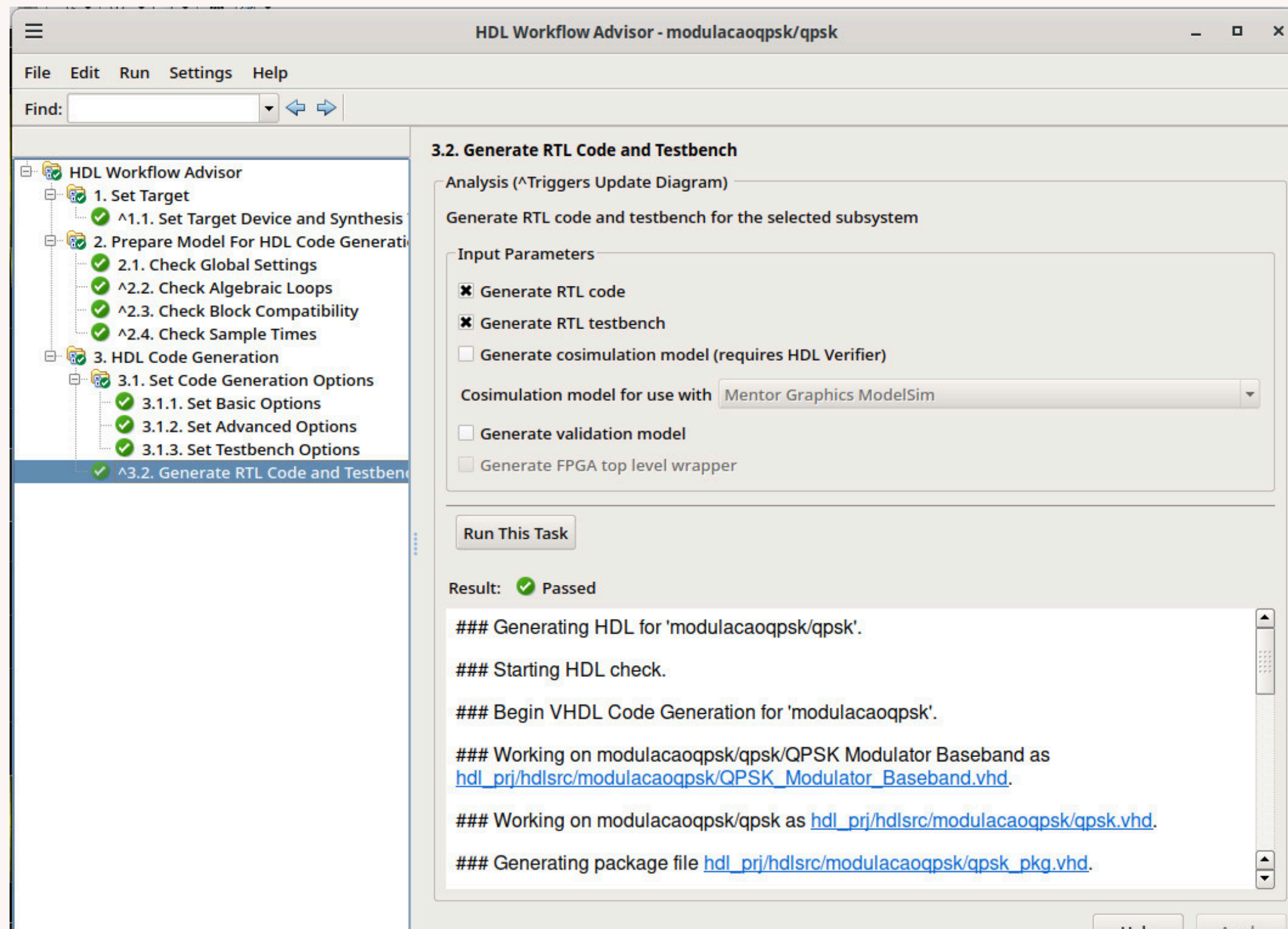


18 - Expanda o Terceiro passo

19 - selecione para gerar o RTL Testbench

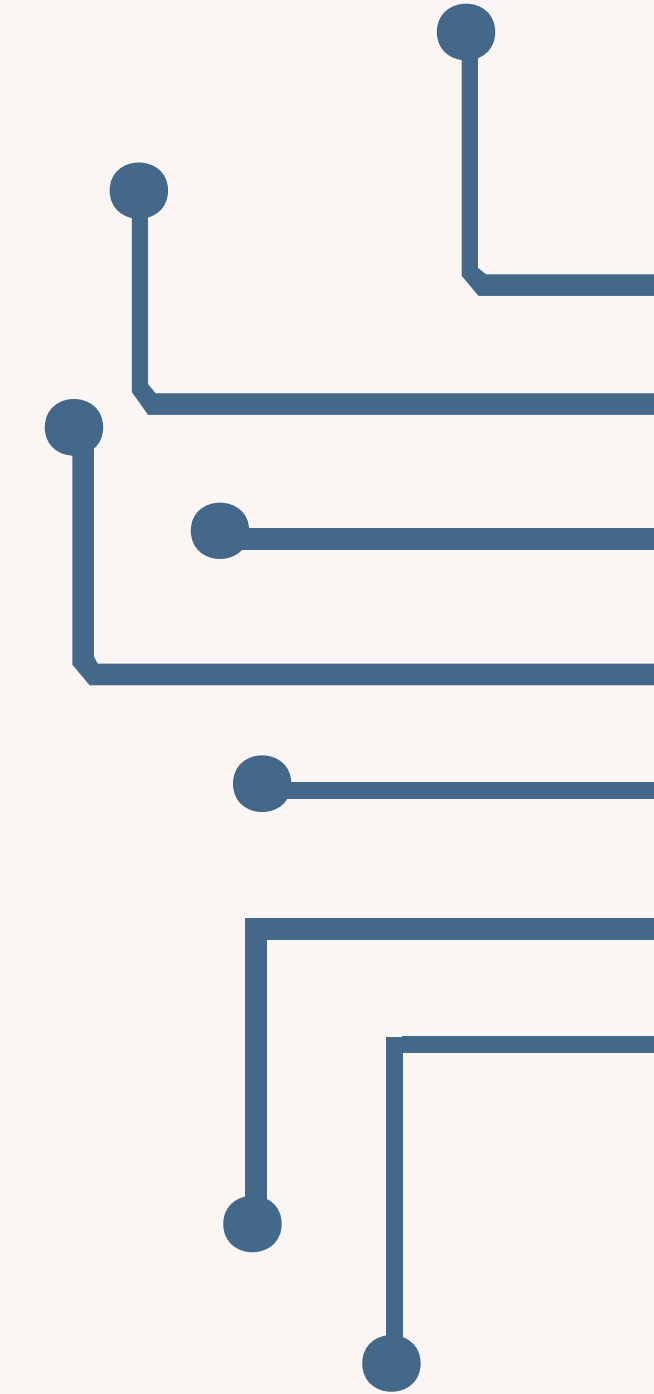
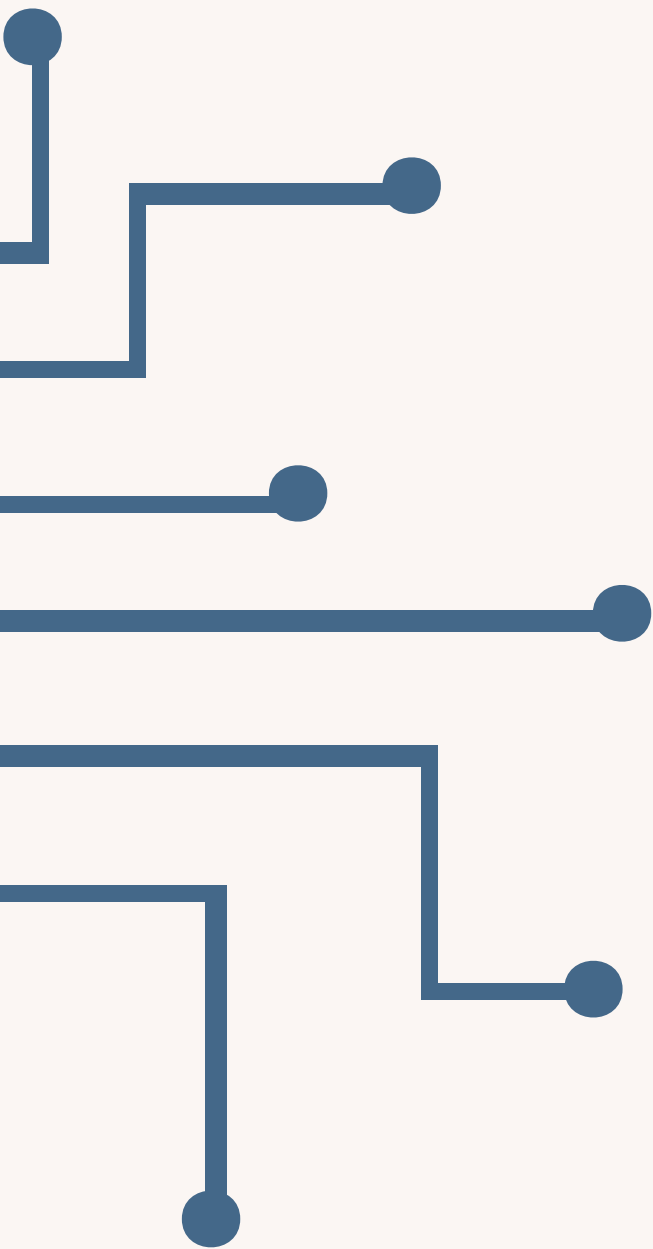


## Desenvolvimento



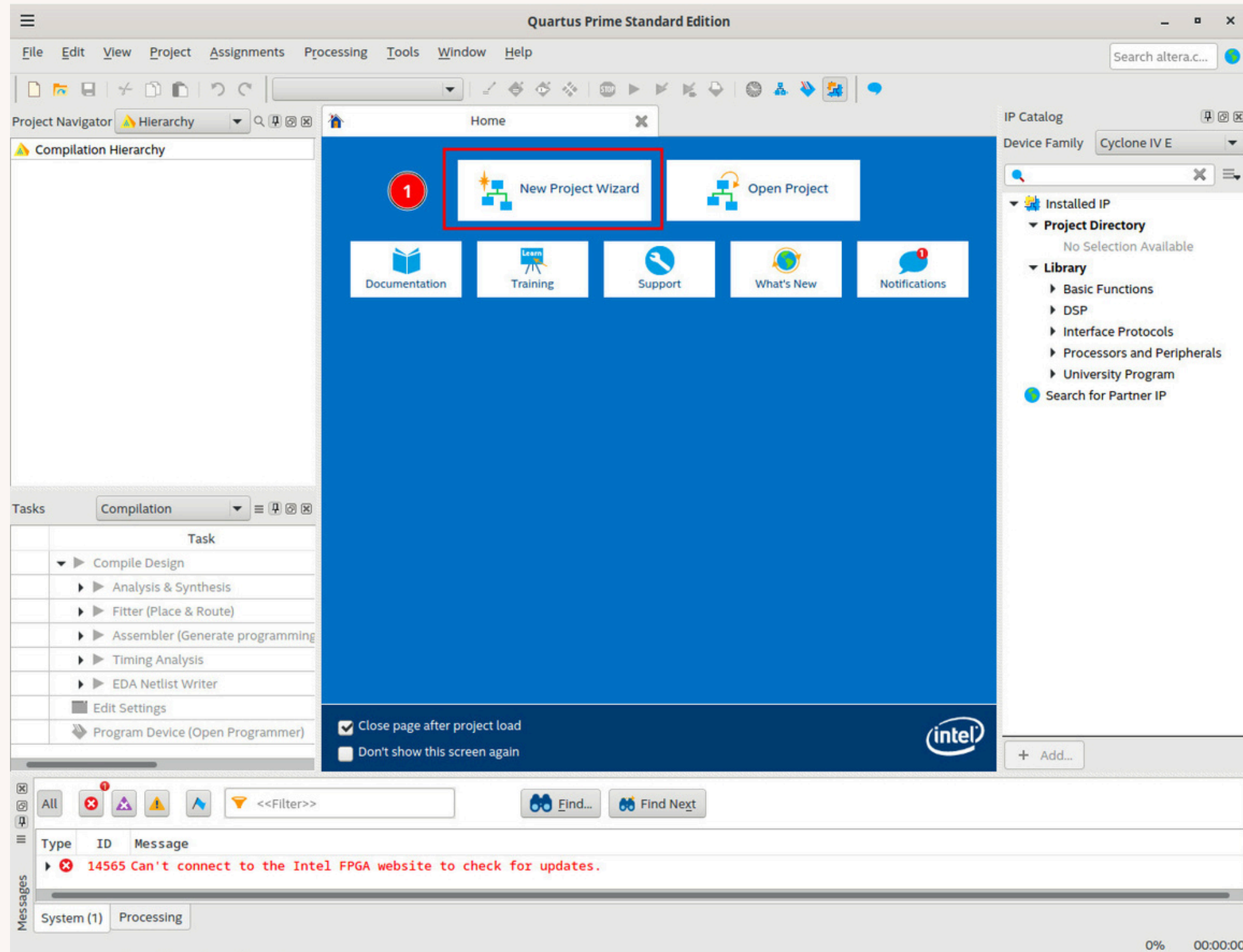
Quando tudo estiver com a indicação “Passed”, o código VHDL foi gerado com sucesso!





Gerado o VHDL,  
vamos para o  
Quartus...

# Desenvolvimento

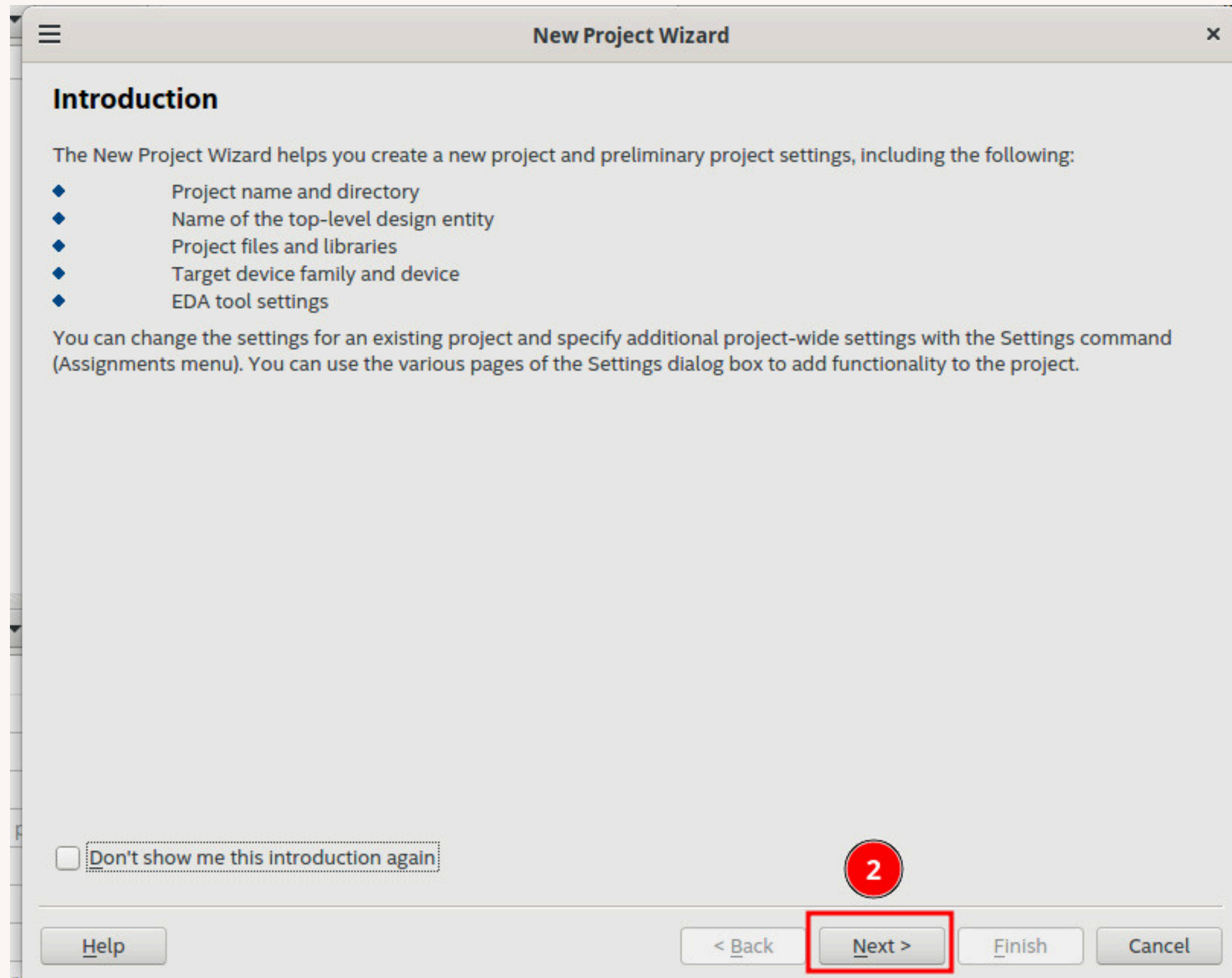


Abra o Quartus

1 - Clique em **New Project Wizard**

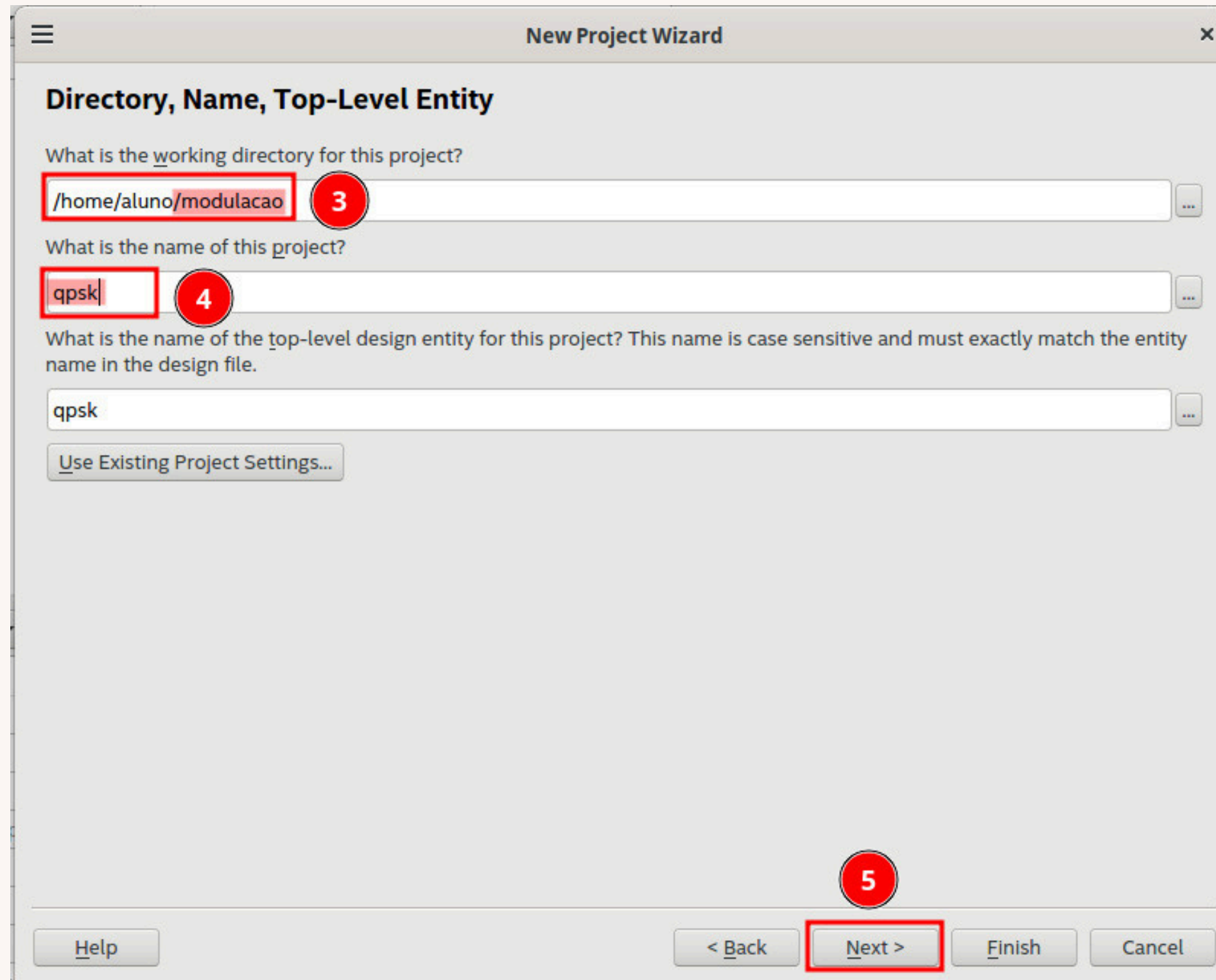


## Desenvolvimento



2 - Clique em **Next >**

## Desenvolvimento



The screenshot shows the 'New Project Wizard' dialog box with the following fields and annotations:

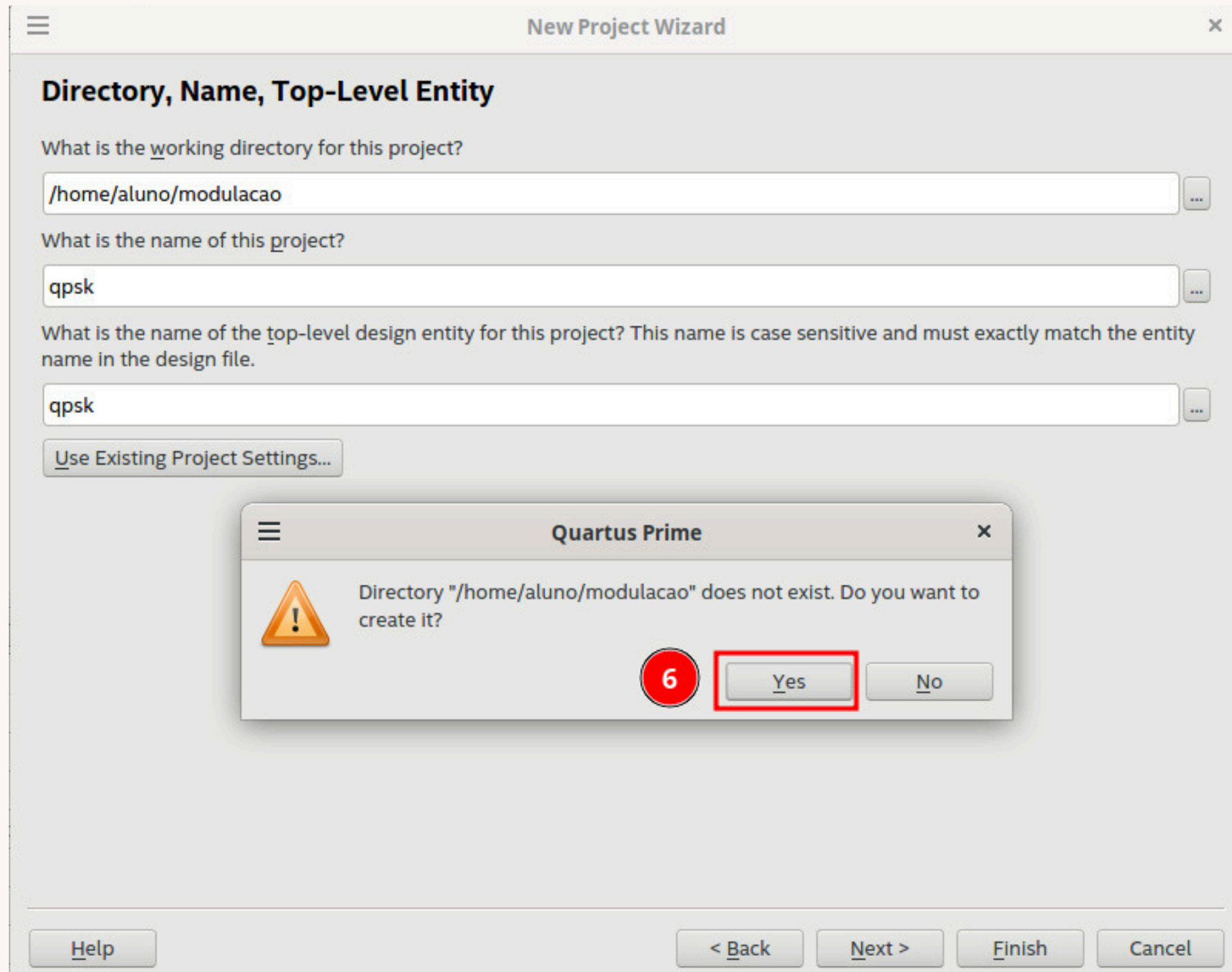
- Directory, Name, Top-Level Entity**
- What is the working directory for this project?  
/home/aluno/modulacao (Annotated with a red box and a red circle with the number 3)
- What is the name of this project?  
qpsk (Annotated with a red box and a red circle with the number 4)
- What is the name of the top-level design entity for this project? This name is case sensitive and must exactly match the entity name in the design file.  
qpsk
- Use Existing Project Settings...
- Navigation buttons at the bottom: Help, < Back, Next > (Annotated with a red box and a red circle with the number 5), Finish, Cancel

3 - Crie o projeto dentro de uma pasta conhecida ou crie uma nova pasta, como fizemos ao criar a pasta **modulacao**

4 - Nomeie o projeto com **qpsk** (nesse caso é necessário que seja esse nome)

5 - clique em **Next>**

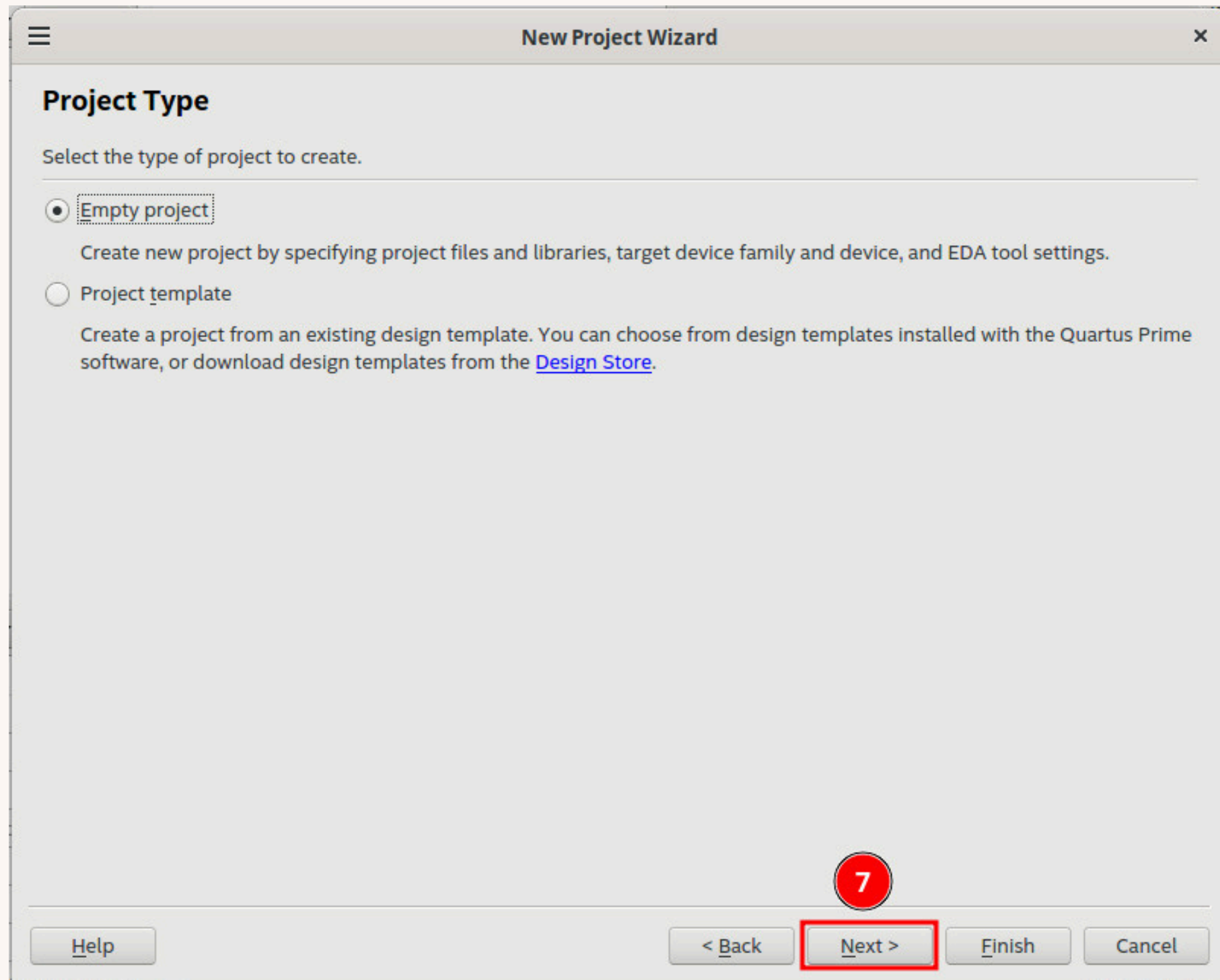
## Desenvolvimento



The image shows a screenshot of the 'New Project Wizard' window in Quartus Prime. The wizard is at the 'Directory, Name, Top-Level Entity' step. It has three input fields: 'What is the working directory for this project?' with the value '/home/aluno/modulacao', 'What is the name of this project?' with the value 'qpsk', and 'What is the name of the top-level design entity for this project?' with the value 'qpsk'. A button 'Use Existing Project Settings...' is visible. Overlaid on this window is a smaller 'Quartus Prime' dialog box with a warning icon and the text: 'Directory "/home/aluno/modulacao" does not exist. Do you want to create it?'. The dialog has two buttons: 'Yes' and 'No'. The 'Yes' button is highlighted with a red rectangle and a red circle containing the number '6'. At the bottom of the wizard window are buttons for '< Back', 'Next >', 'Finish', and 'Cancel'. A 'Help' button is also present in the bottom left corner.

6 - Caso a pasta não exista, será perguntado se gostarias de criá-la, clique em **Yes**

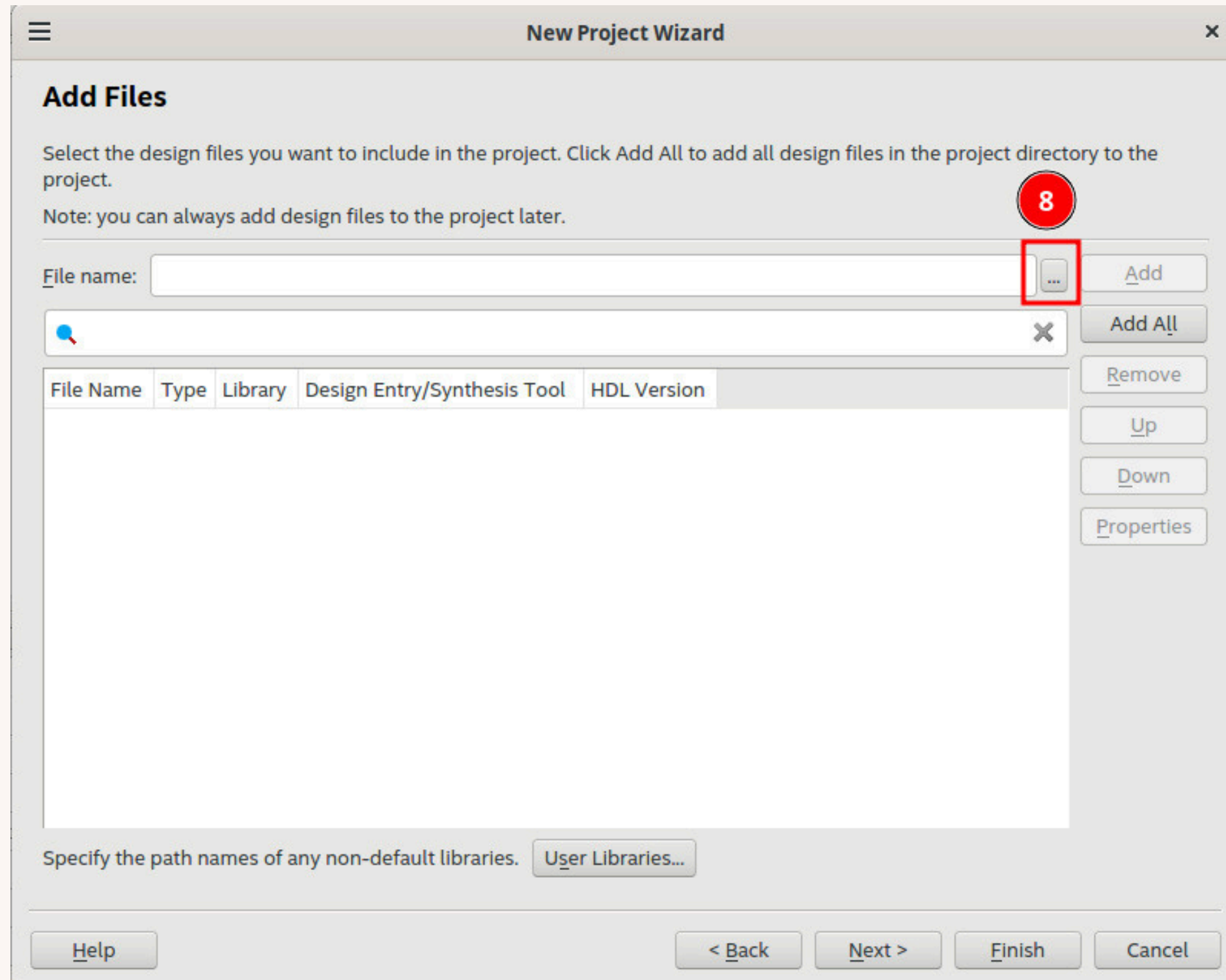
## Desenvolvimento



Crie um projeto vazio (opção padrão)

7 - Selecione **Next >**

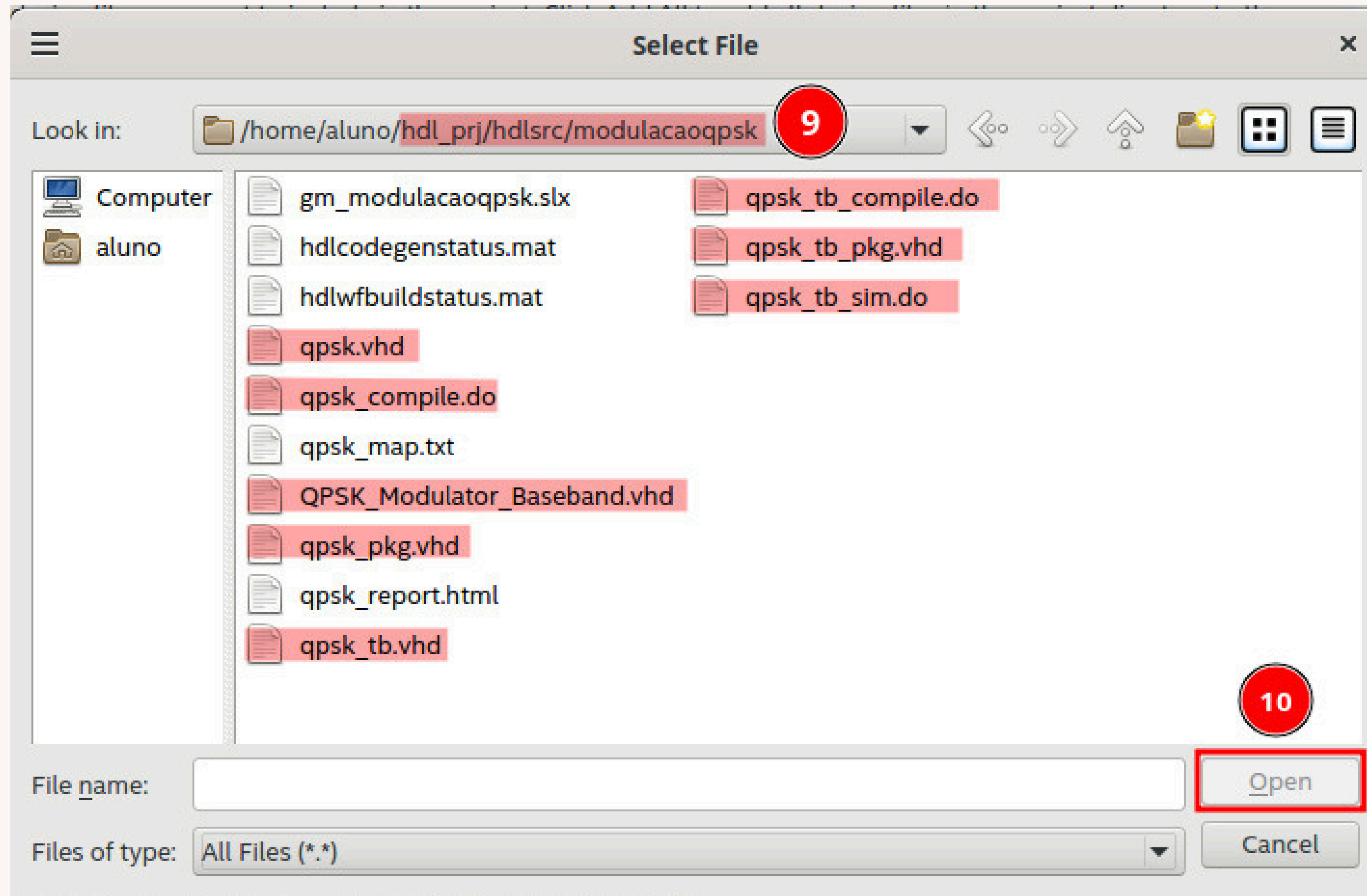
## Desenvolvimento



8 - Clique nos “três pontinhos”, para adicionar os arquivos



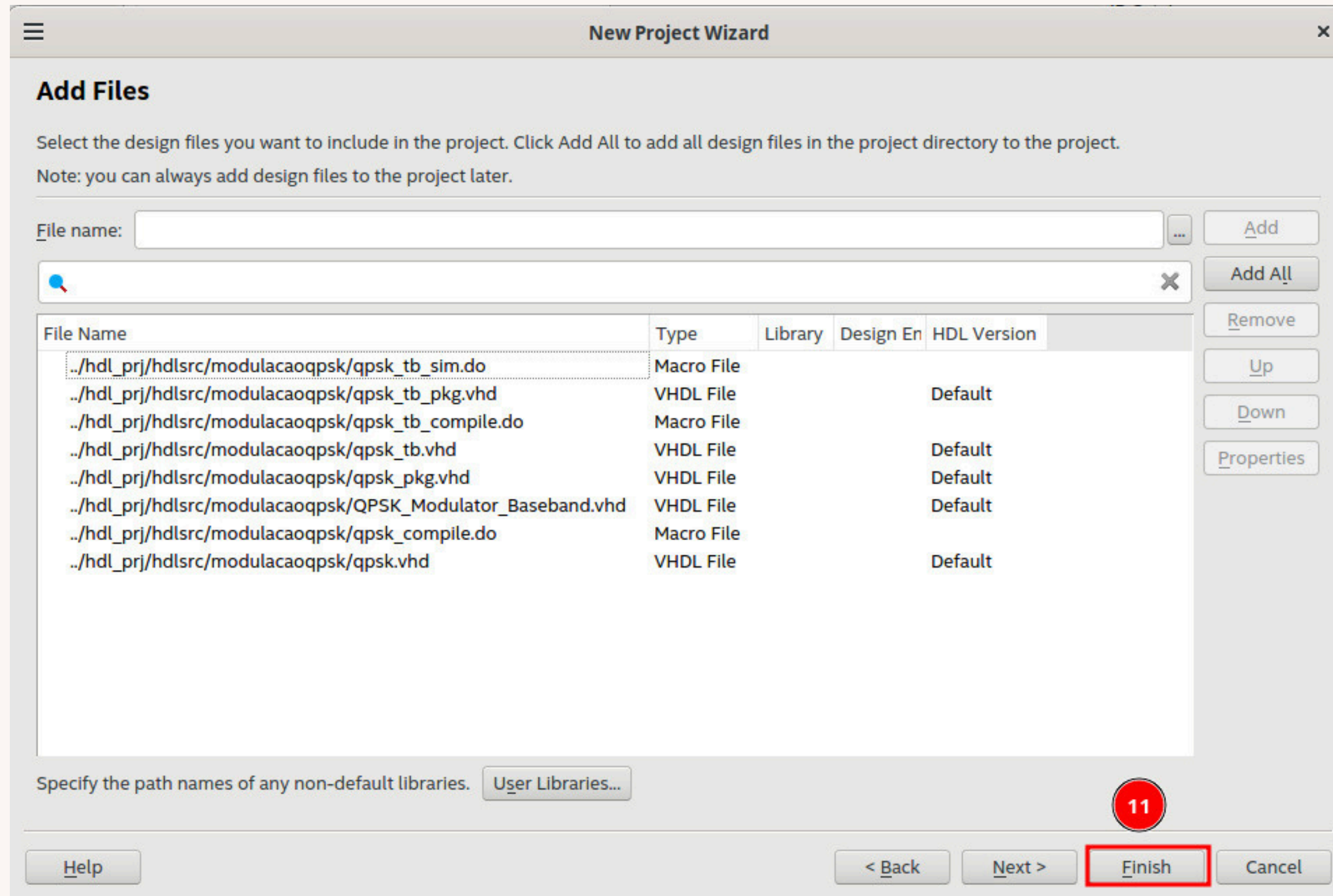
## Desenvolvimento



9 - Selecione os arquivos com extensões **.vhd** e **.do** do diretório que o simulink guardou o projeto

10 - Clique em **Open**

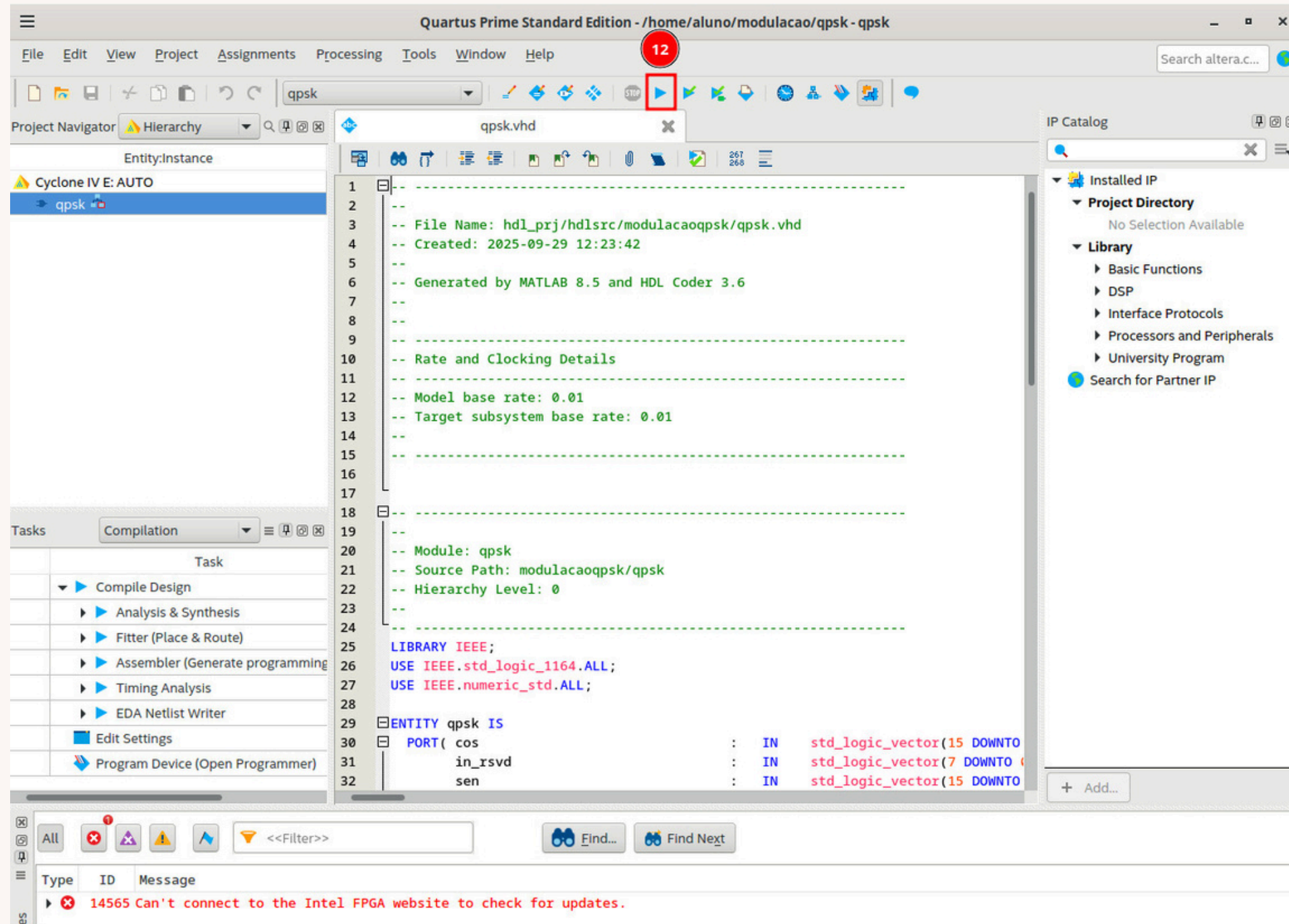
## Desenvolvimento



Abrirá outra aba...

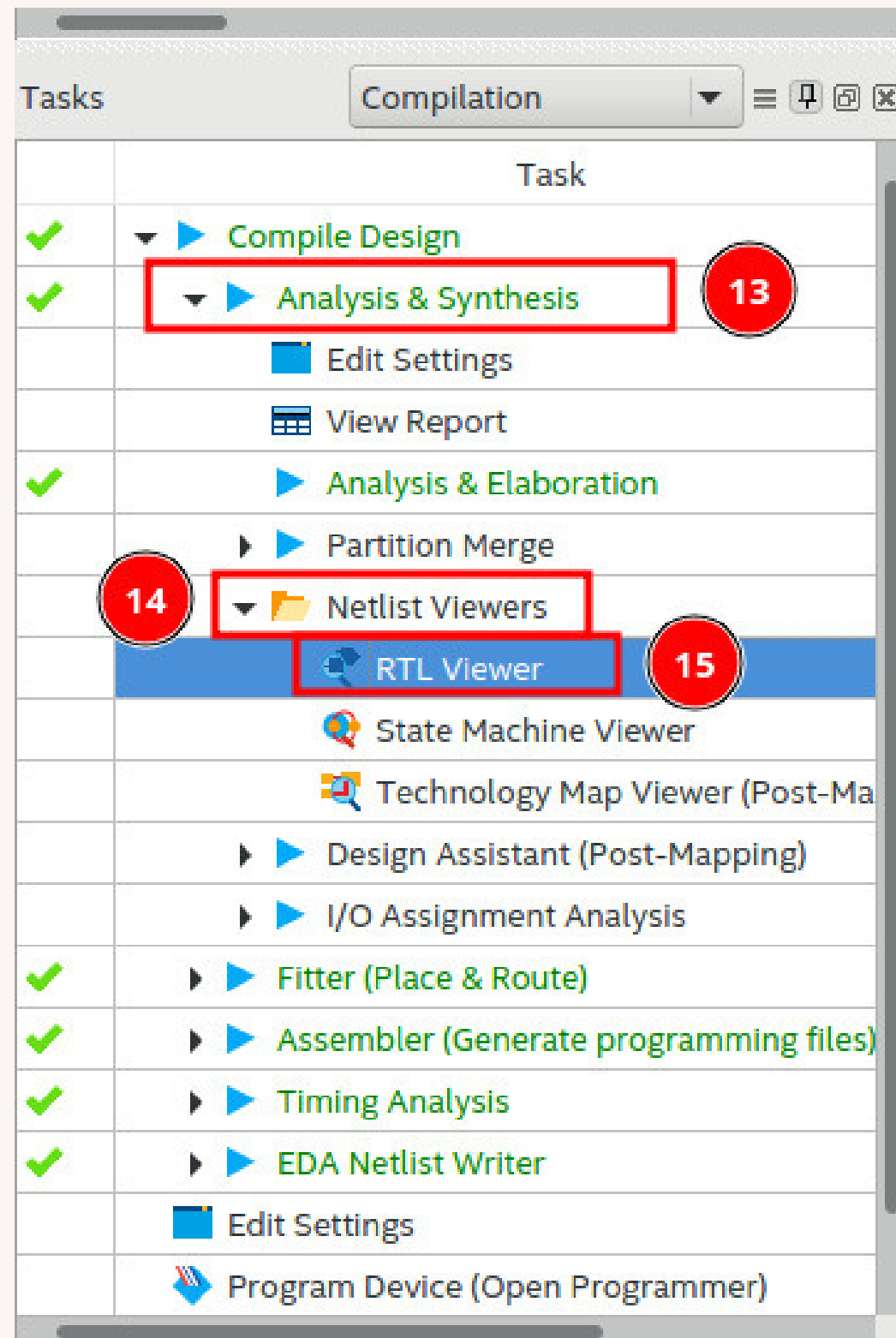
11 - Clique em **Finish**

## Desenvolvimento



12 - Compile o projeto no ícone destacado

## Desenvolvimento



No canto inferior esquerdo, aparecerão alguns ícones de verificação verdes..

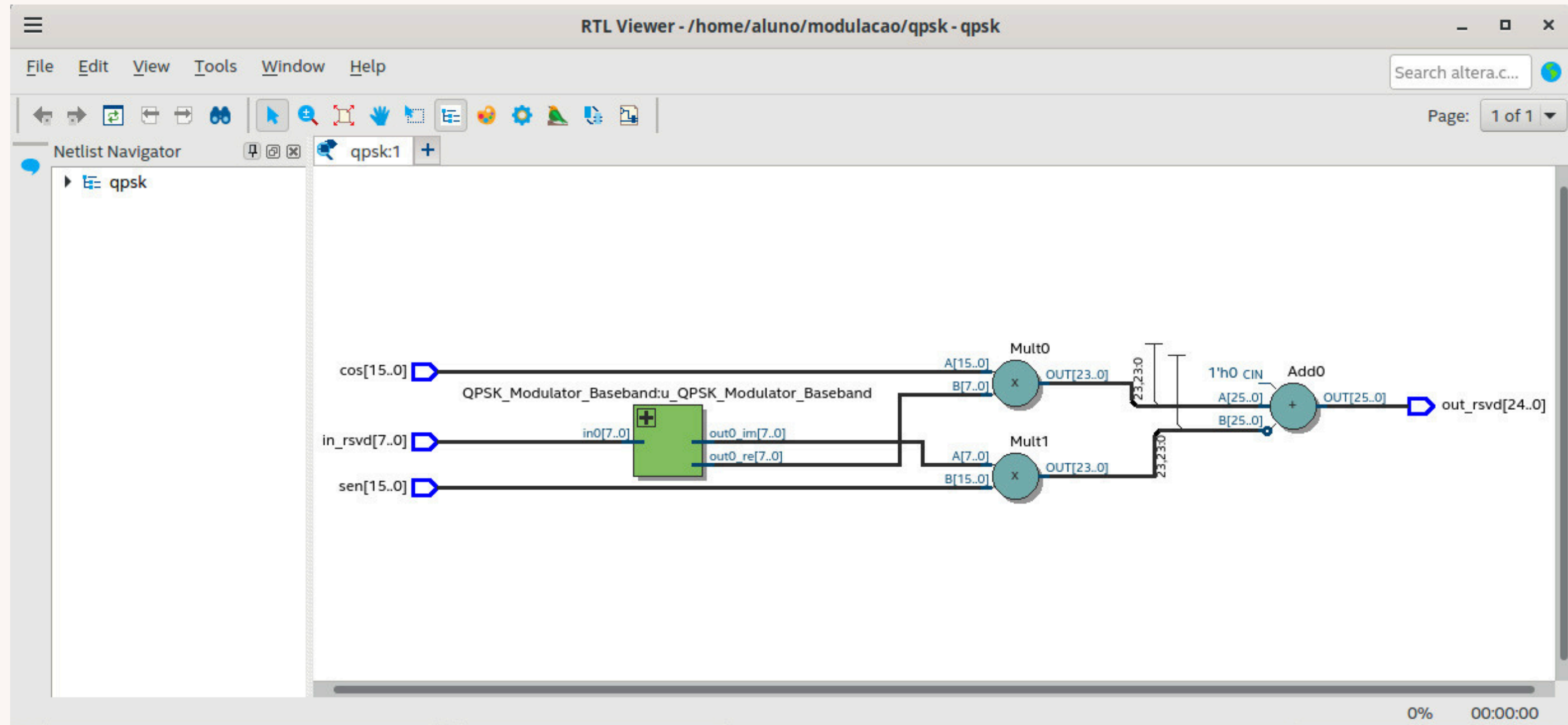
13 - Clique em **Analysis & Synthesis**

14 - Dê um duplo clique em **Netlist Viewers**

15 - Clique em **RTL Viewer**



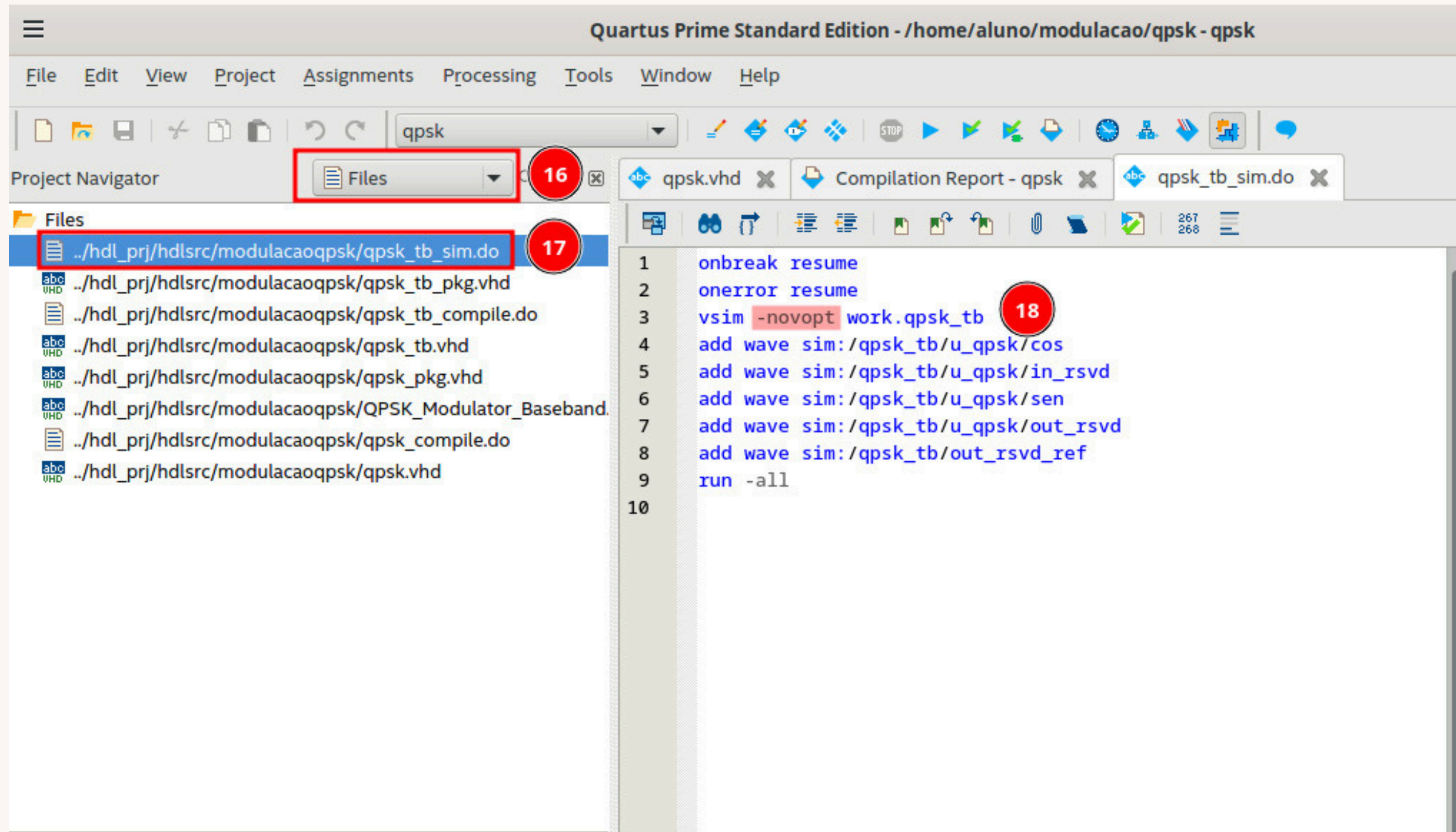
## Desenvolvimento



Abrirá essa aba. Note que se assemelha ao sistema do simulink (página 17)



## Desenvolvimento

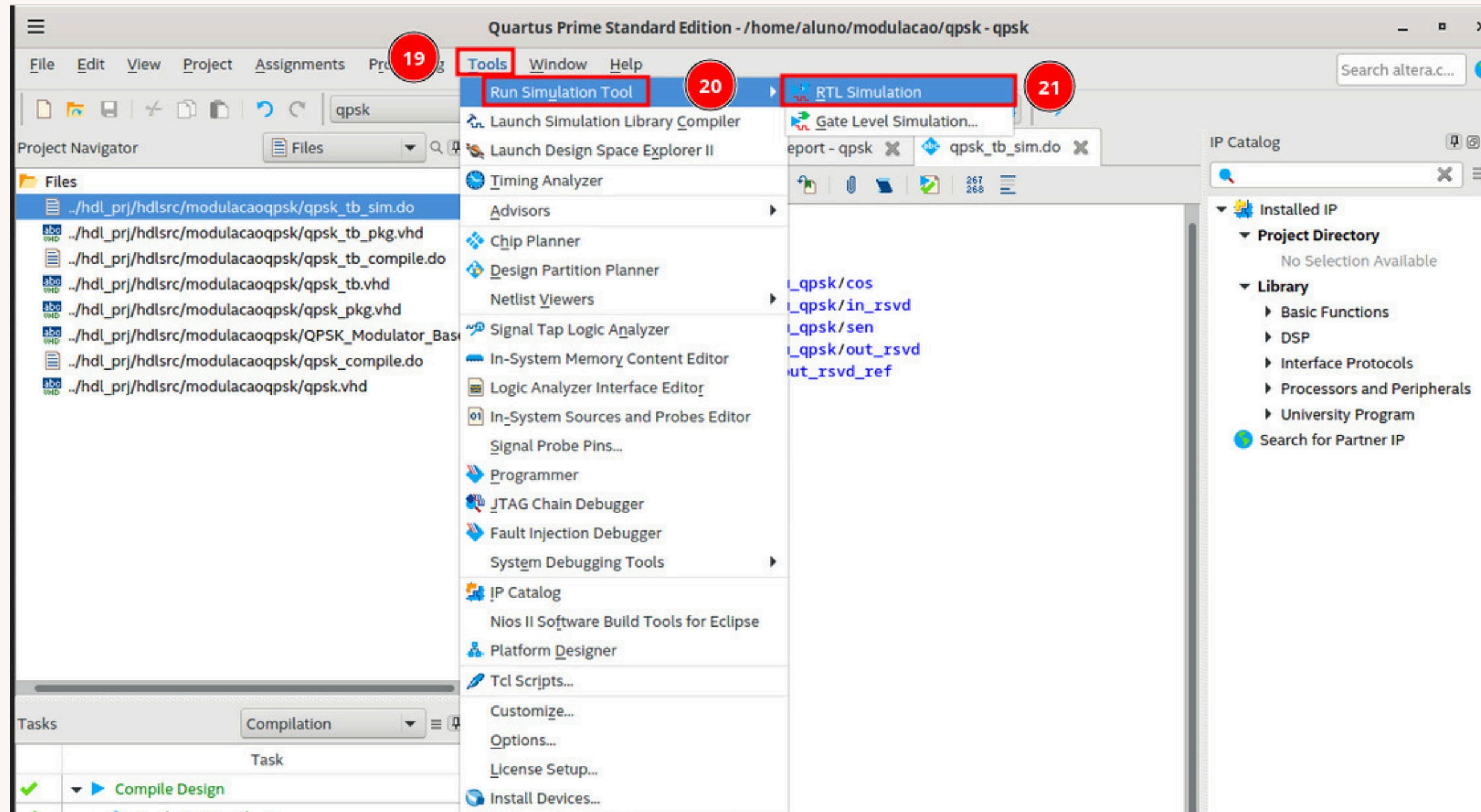


16 - Troque de Hierarchy para **Files**

17 - Clique no arquivo **qpsk\_tb\_sim.do**

18 - Apague a parte destacada e Salve com **Ctrl+S**

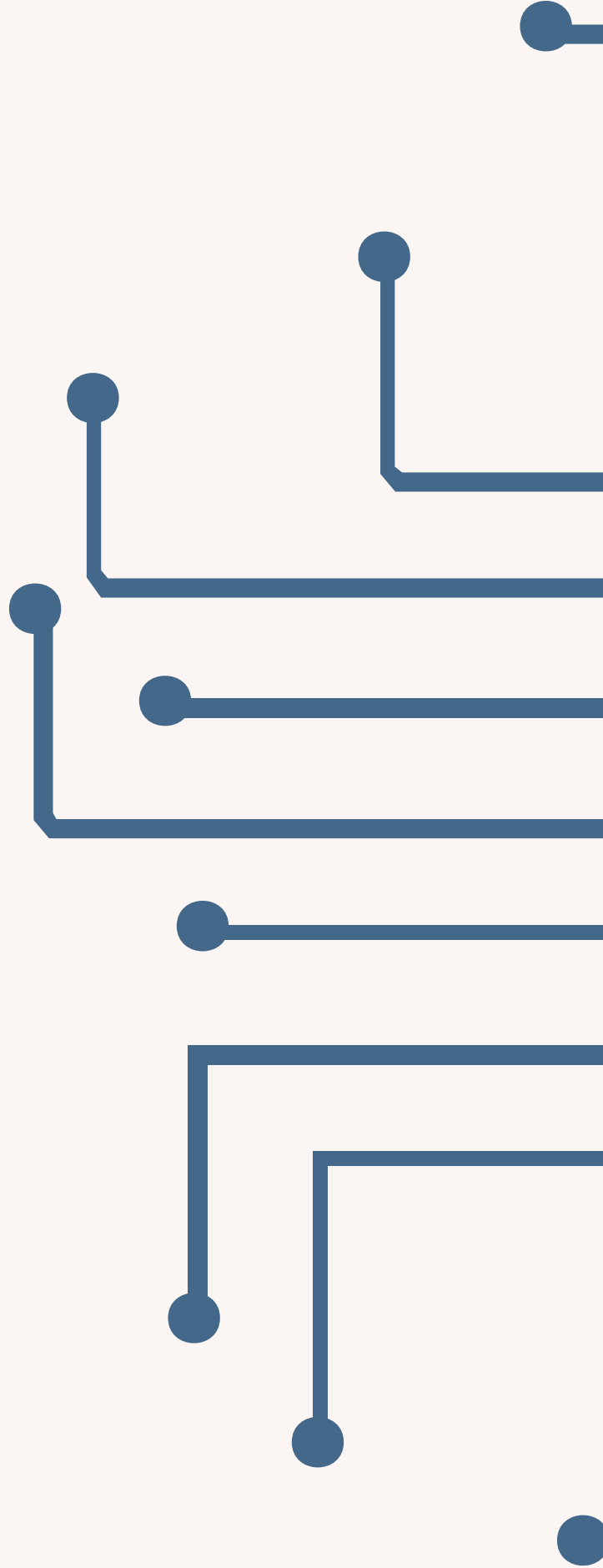
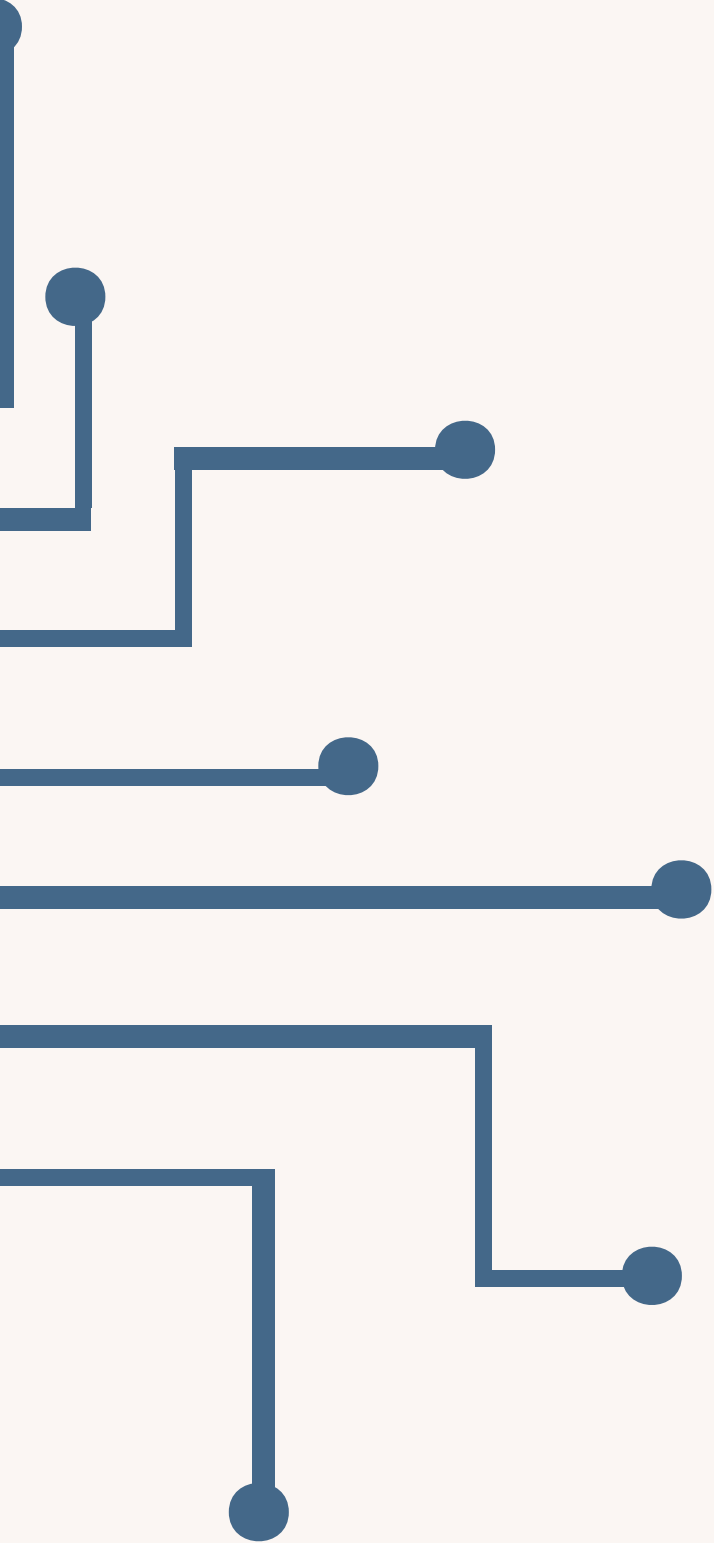
## Desenvolvimento



19 - Clique em **Tools**

20 - Clique em **Run Simulation Tool**

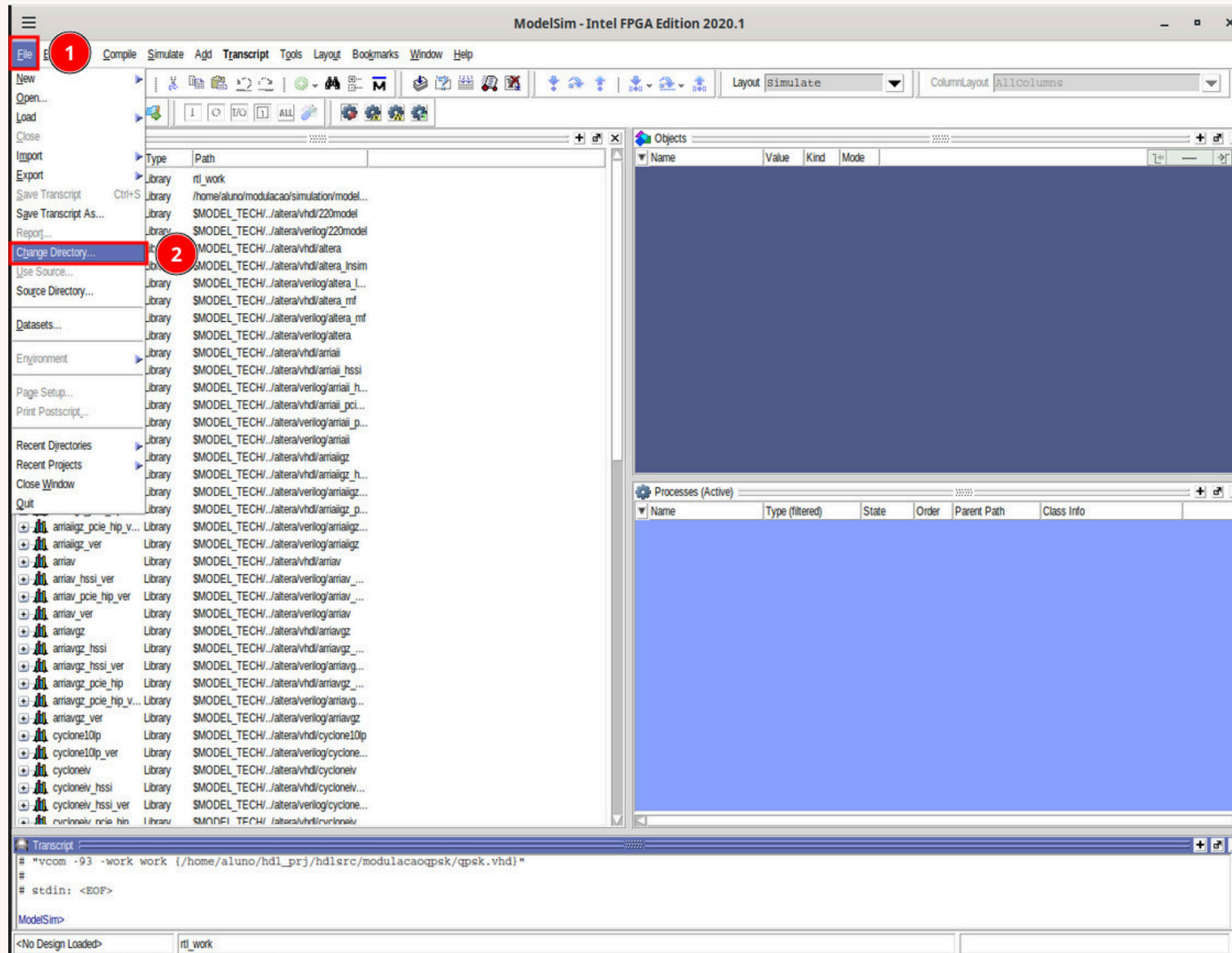
21 - Clique em **RTL Simulation**



Abrirá o  
Modelsim...



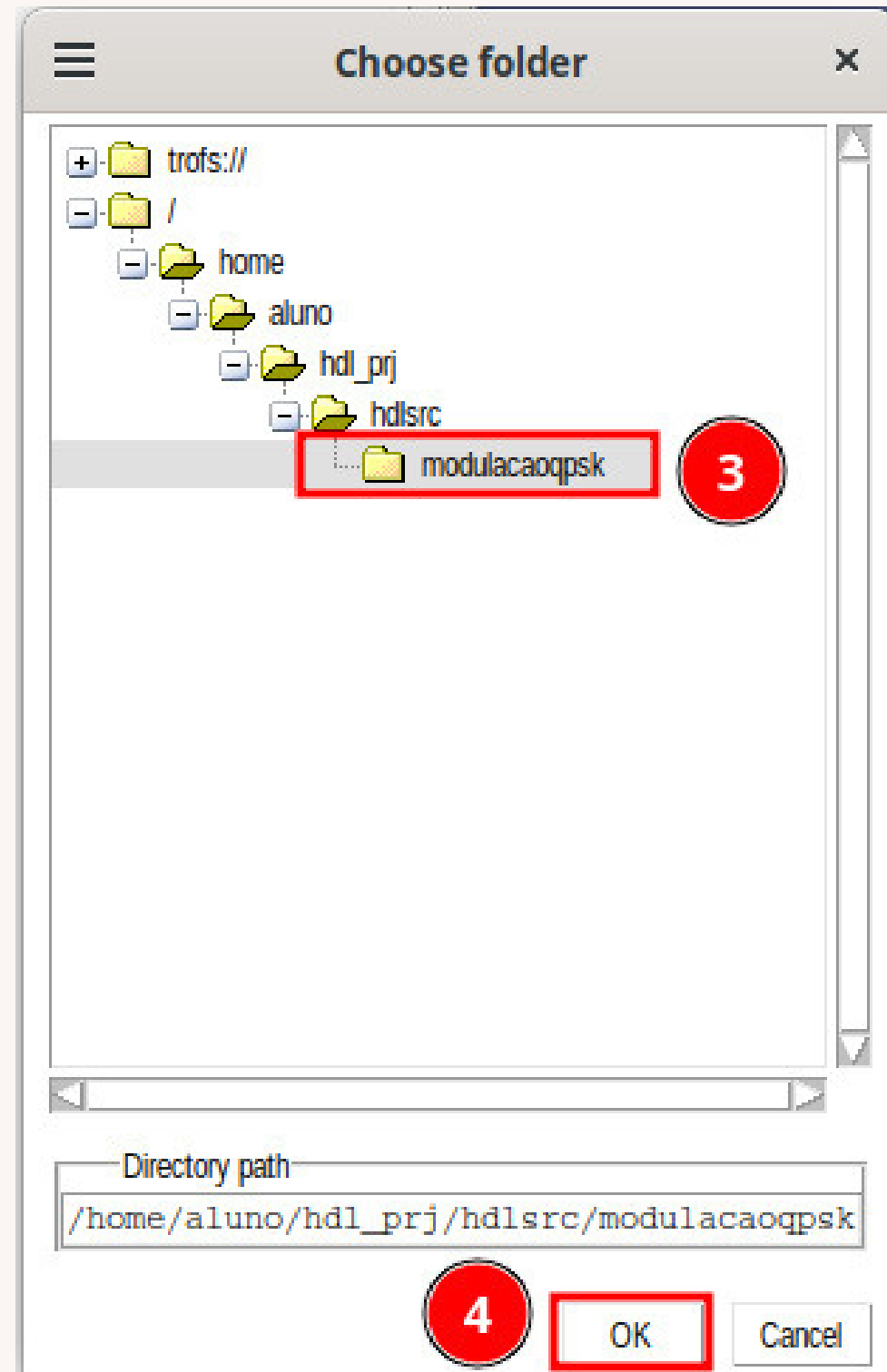
# Desenvolvimento



1 - Clique em **File**

2 - Clique em **Change Directory...**

## Desenvolvimento

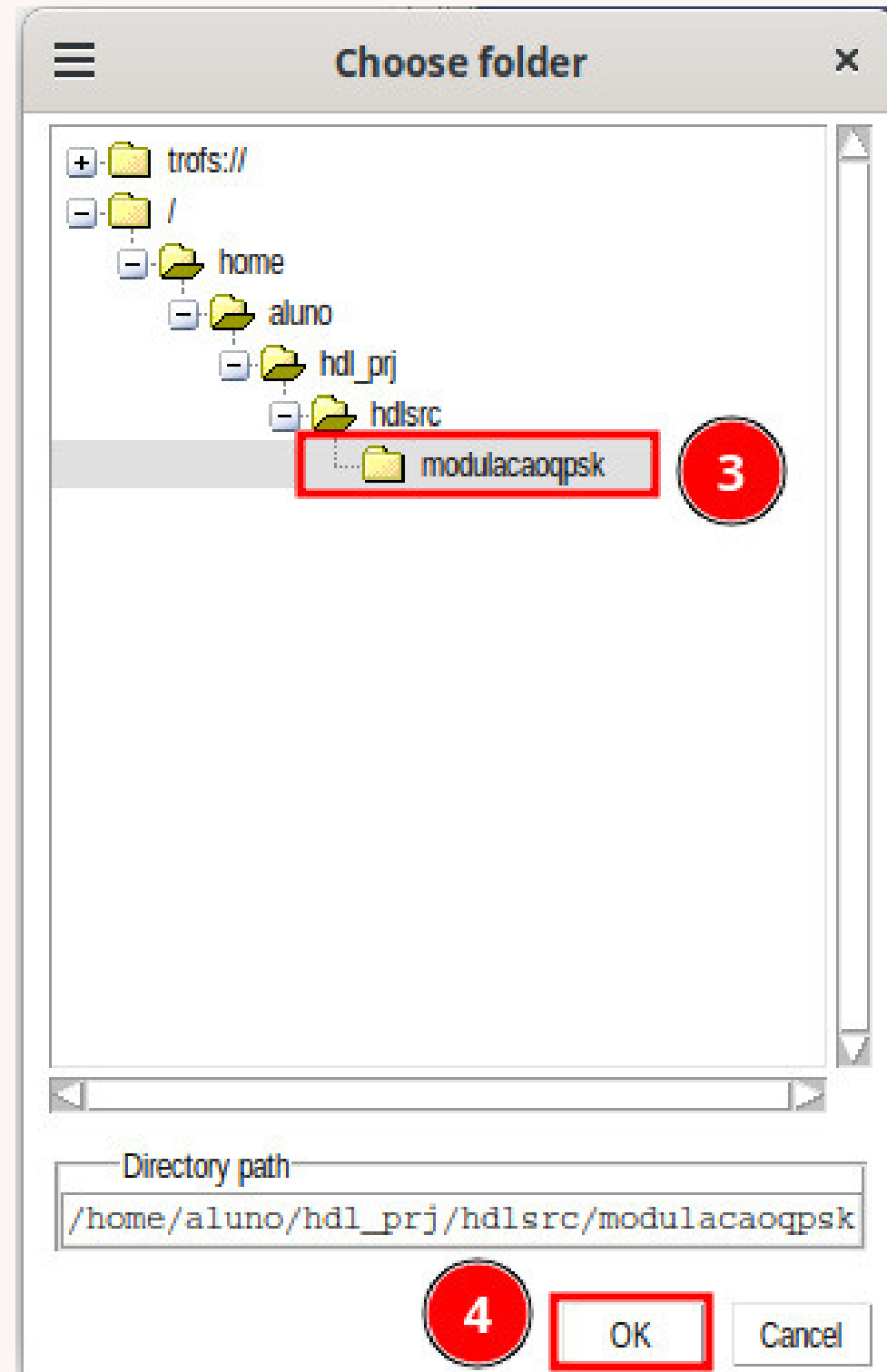


3 - Abra no projeto criado pelo simulink

4- Clique em **Ok**



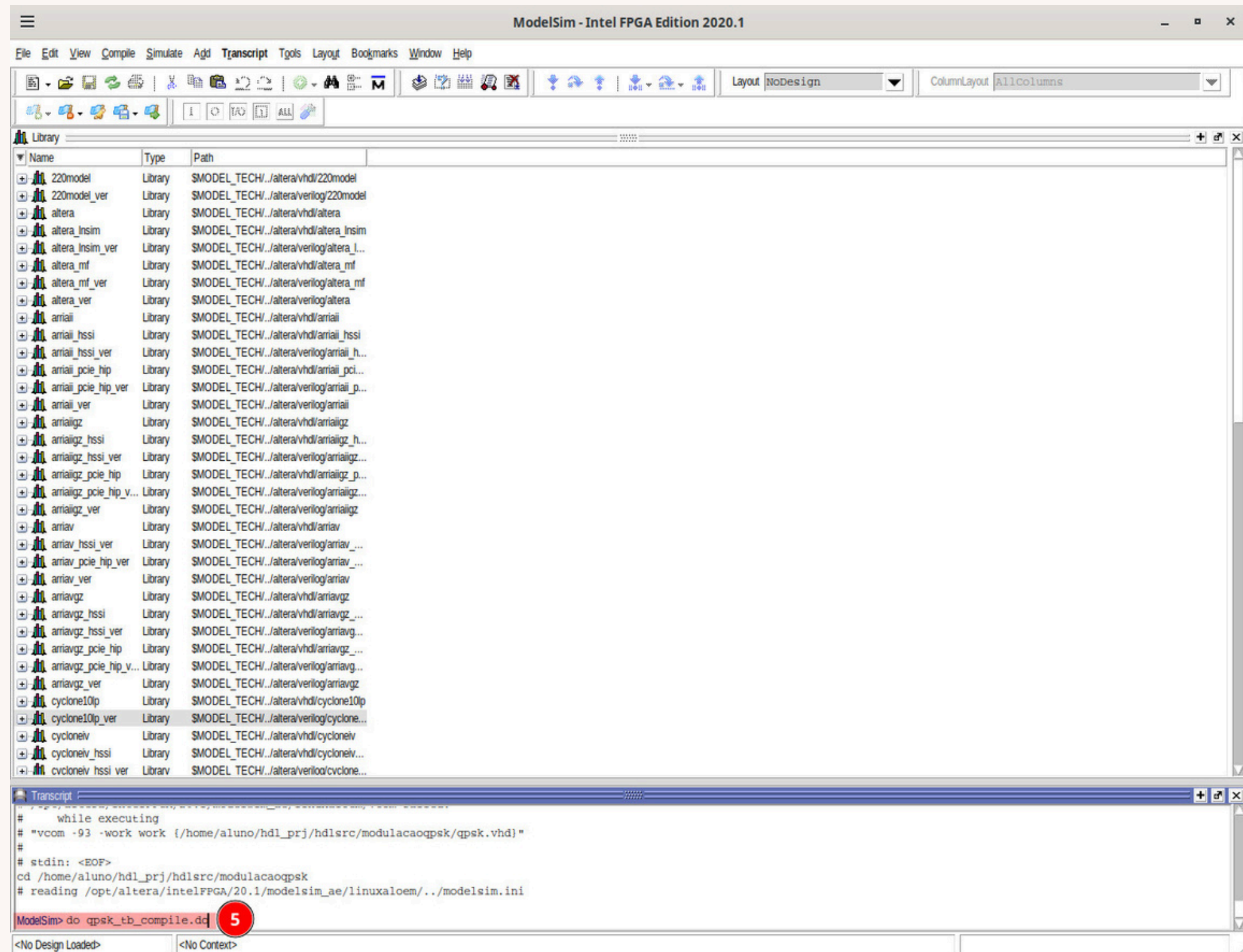
## Desenvolvimento



3 - Abra no projeto criado pelo simulink

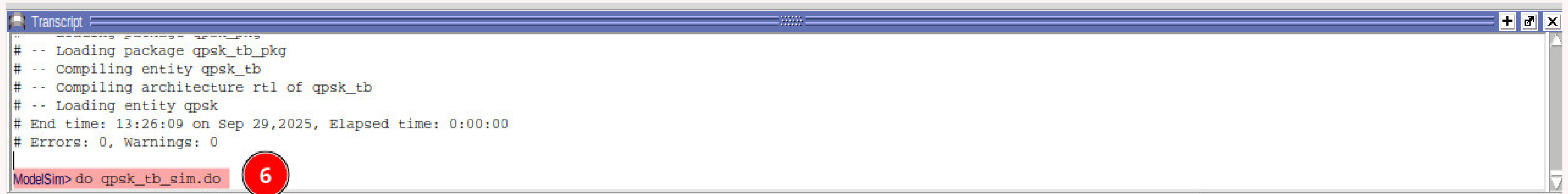
4- Clique em **Ok**

# Desenvolvimento



5 - Dê o comando:  
**do qpsk\_tb\_compile.do**

## Desenvolvimento



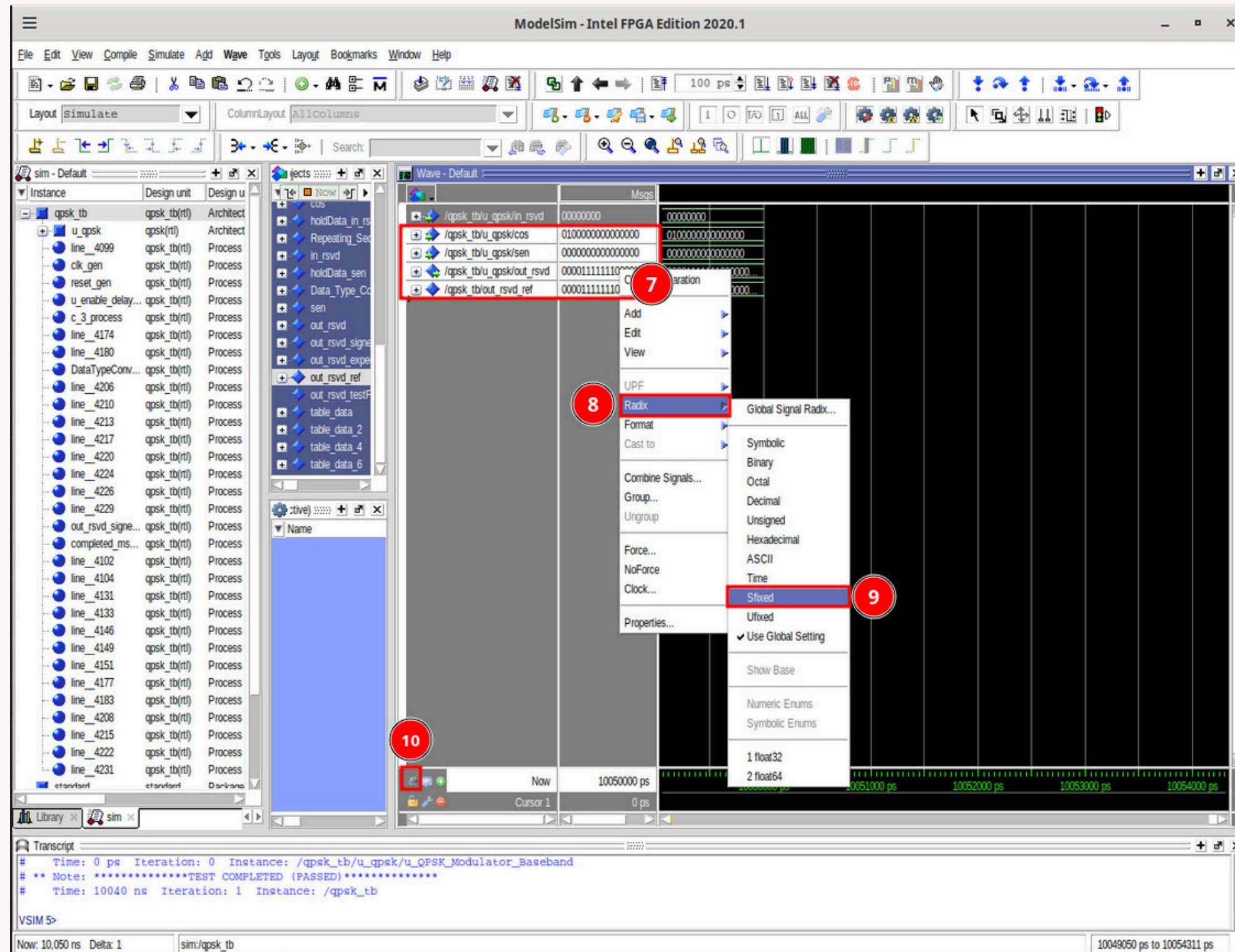
The screenshot shows a ModelSim Transcript window with the following text:

```
# -- Loading package qpsk_tb_pkg  
# -- Compiling entity qpsk_tb  
# -- Compiling architecture rtl of qpsk_tb  
# -- Loading entity qpsk  
# End time: 13:26:09 on Sep 29,2025, Elapsed time: 0:00:00  
# Errors: 0, Warnings: 0  
ModelSim> do qpsk_tb_sim.do
```

A red circle with the number 6 is positioned next to the command prompt line.

6 - Dê o comando:  
**do qpsk\_tb\_sim.do**

## Desenvolvimento



7 - Selecione todos os sinais, exceto o **in\_rsvd**, e clique com o botão direito.

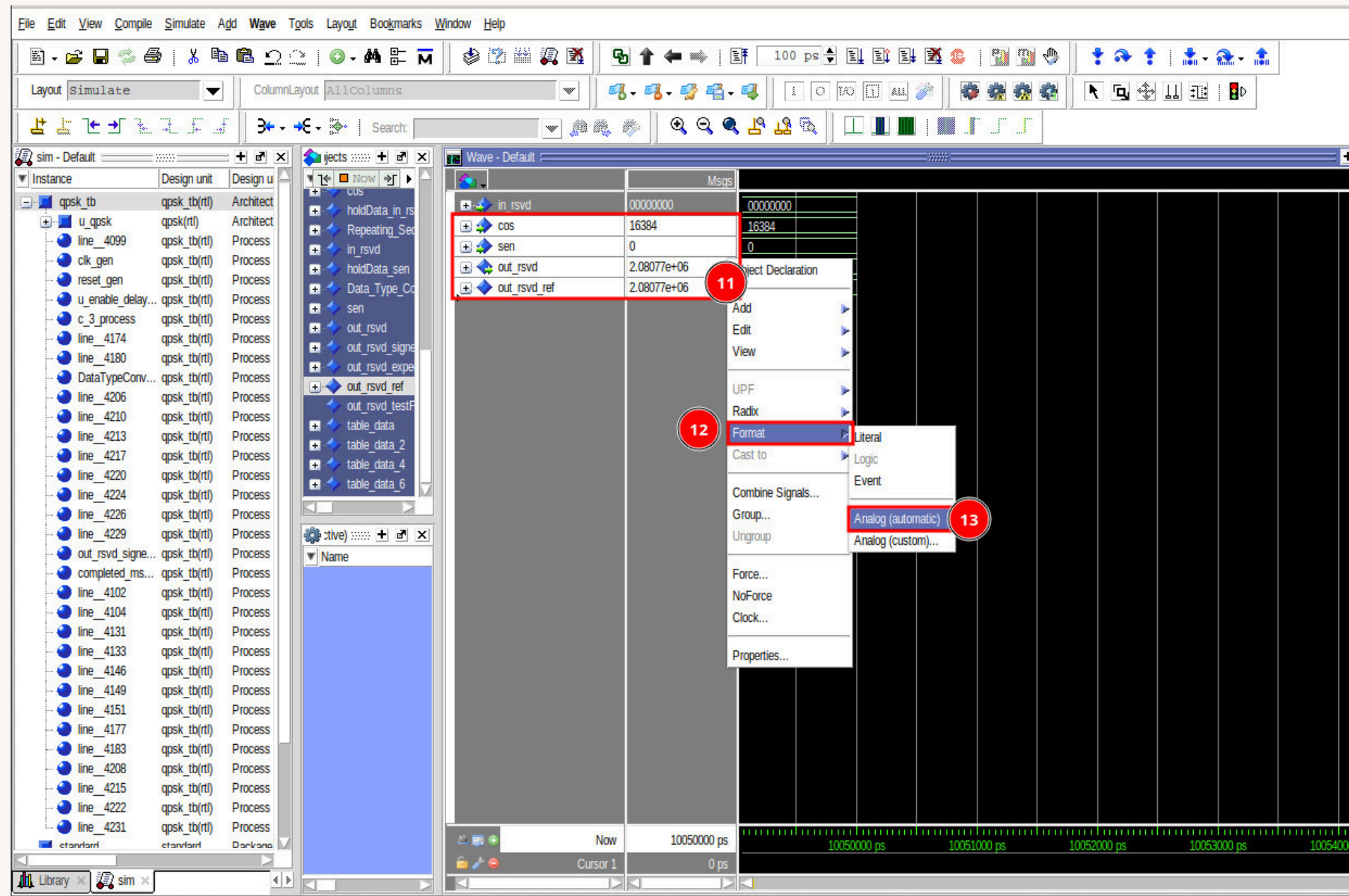
8 - Selecione **Radix**

9 - Escolha **Sfixed**

10 - Clique no ícone indicado para melhor visualização



## Desenvolvimento



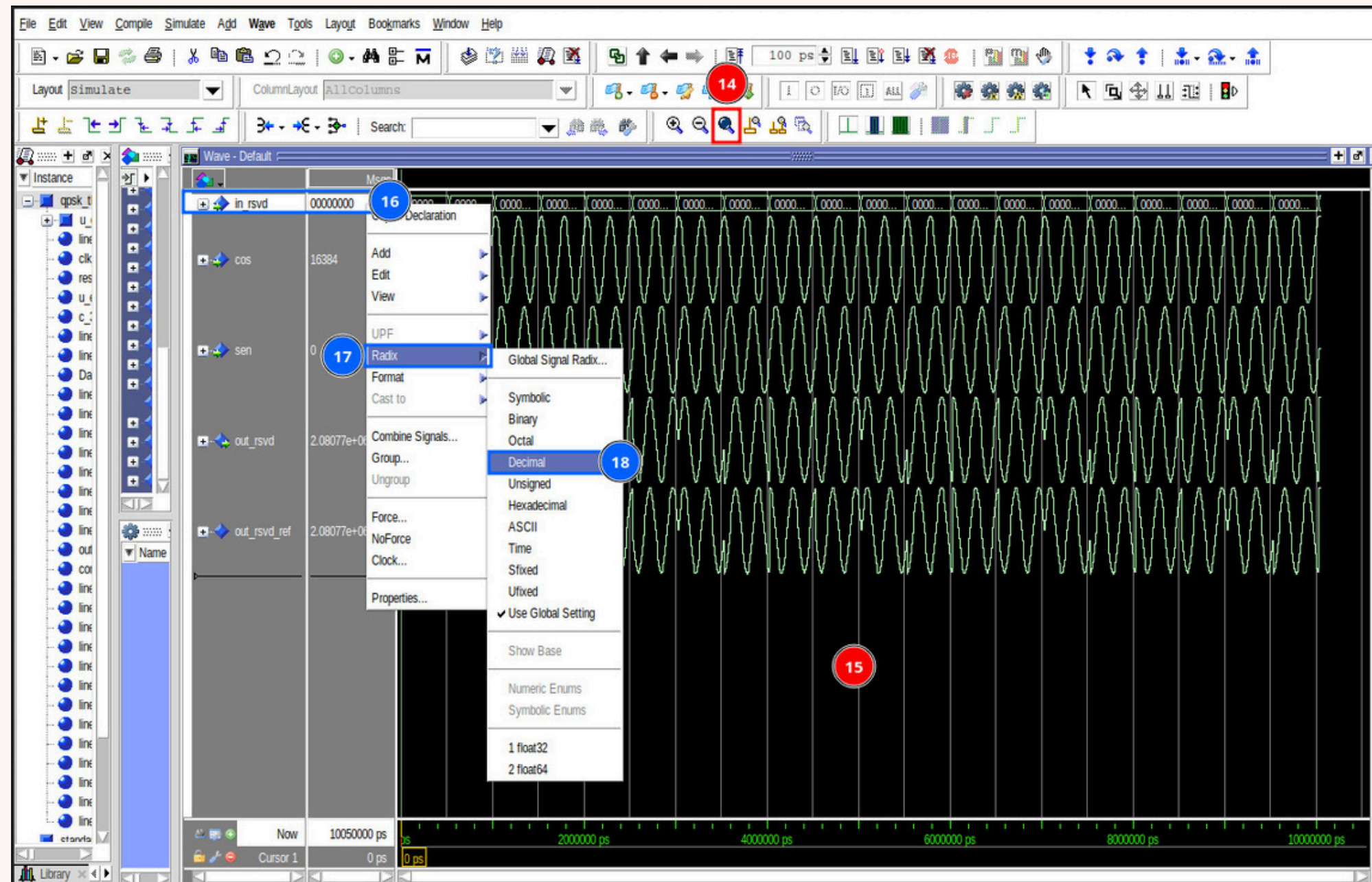
11 - Selecione novamente os mesmos sinais e clique com o botão direito

12 - Selecione **Format**

13 - Escolha **Analog(automatic)**



## Desenvolvimento



14 - Clique na lupa indicada

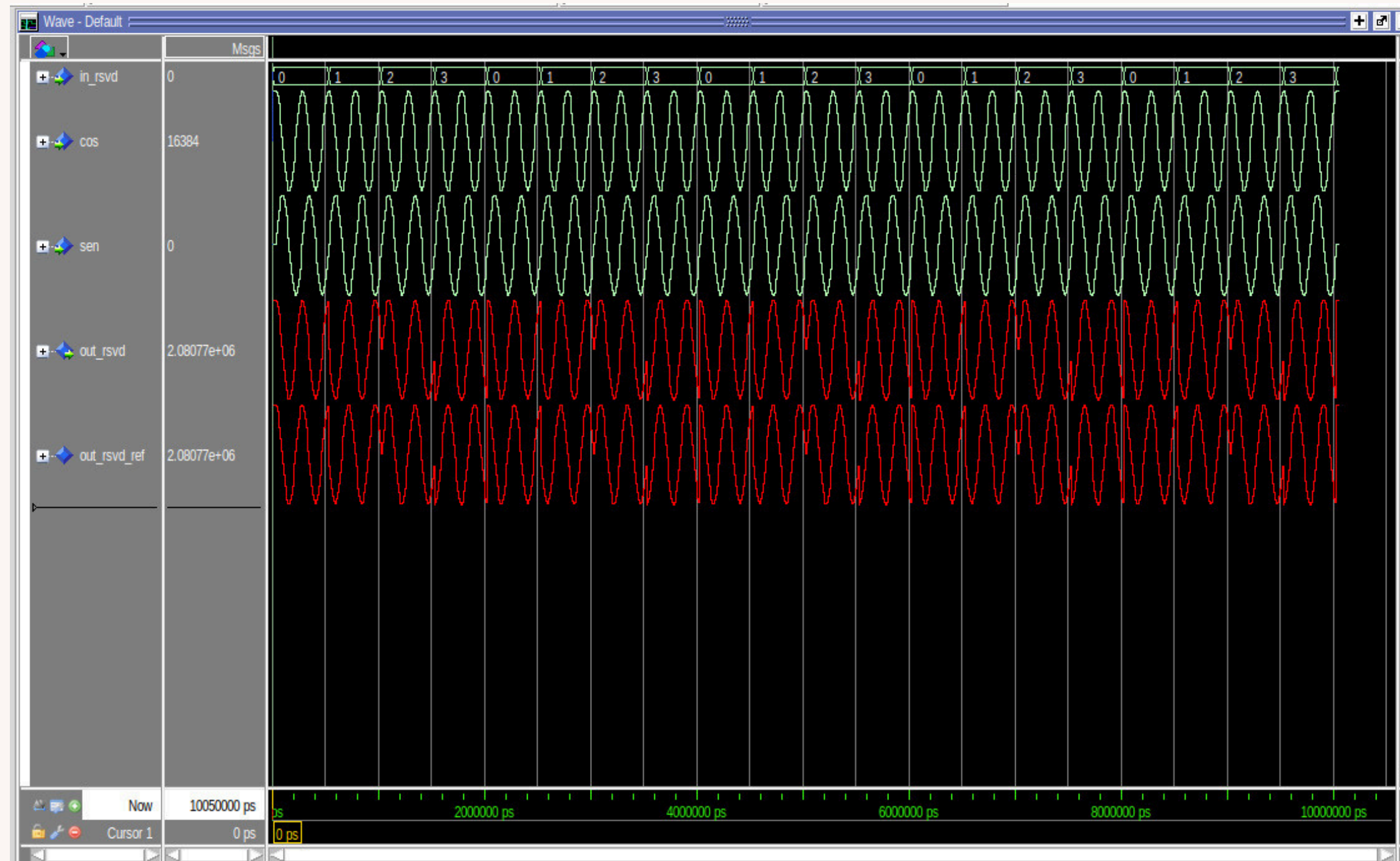
15 - Clique em qualquer lugar dentro da wave

16 - Selecione o sinal **in\_rsvd**

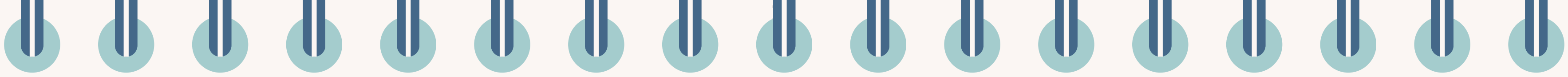
17 - Selecione **Radix**

18 - Escolha **Decimal**

## Desenvolvimento



A saída pode ser destacada com uma cor diferente para melhor visualização.



## Desenvolvimento

Pronto! Agora temos uma modulação QPSK, em que a entrada assume os valores 0, 1, 2 ou 3, cada um representando dois bits, provocando uma mudança de fase do sinal de  $0^\circ$ ,  $90^\circ$ ,  $180^\circ$  ou  $270^\circ$





**Muito  
obrigado!**